

PoneglyphDB: Efficient Non-interactive Zero-Knowledge Proofs for Arbitrary SQL-Query Verification

Binbin Gu
University of California, Irvine
Irvine, USA
binbing@uci.edu

Juncheng Fang
University of California, Irvine
Irvine, USA
junchf1@uci.edu

Faisal Nawab
University of California, Irvine
Irvine, USA
nawabf@uci.edu

Abstract

In database applications involving sensitive data, the dual imperatives of data confidentiality and provable (verifiable) query processing are important. This paper introduces PoneglyphDB, a database system that leverages non-interactive zero-knowledge proofs (ZKP) to support both confidentiality and provability. Unlike traditional databases, PoneglyphDB enhances confidentiality by ensuring that raw data remains exclusively with the host, while also enabling verifying the correctness of query responses by providing proofs to clients.

The main innovation in this paper is proposing efficient ZKP designs (called circuits) for basic operations in SQL query processing. These basic operation circuits are then combined to form ZKP circuits for larger, more complex queries. PoneglyphDB's circuits are carefully designed to be efficient by utilizing advances in cryptography such as PLONKish-based circuits, recursive proof composition techniques, and designing with low-order polynomial constraints. We demonstrate the performance of PoneglyphDB with the standard TPC-H benchmark. Our experimental results show that PoneglyphDB can efficiently achieve both confidentiality and provability, outperforming existing state-of-the-art ZKP methods.

Keywords

Zero-Knowledge Proofs, Verifiable SQL Queries

ACM Reference Format:

Binbin Gu, Juncheng Fang, and Faisal Nawab. 2025. PoneglyphDB: Efficient Non-interactive Zero-Knowledge Proofs for Arbitrary SQL-Query Verification. In *Proceedings of the International Conference on Management of Data (SIGMOD'25)*. ACM, Berlin, Germany, 15 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

As databases serve as the backbone of diverse applications, the responsibility entrusted to them extends beyond data storage—it encompasses the safeguarding of sensitive information. This is an important consideration, especially for databases that store personal data. Also, in applications with sensitive information such as census data or unemployment statistics, it is important for users to trust that the database owner is using the correct database (i.e., one that is consistent with

the claimed database) and that the provided answers accurately reflect the intended computations.

We tackle the problem of providing a database solution to ensure two characteristics: (1) **Confidentiality**: this property means that raw data is only maintained at the host (i.e., service provider) and is not shared or made public to any other node. Other nodes only receive responses to specific queries that they send to the host node and that the host node agrees to process. (2) **Provability**: ensuring the correctness of processing. This means that the results computed by the host node and sent back to users reflect the correct processing of the query on the private database. This requires that the host node provides a proof of the correctness of the query response to the user.

As an example, consider the healthcare sector, where medical research institutions collect patient data for collaborative studies. Institution X may wish to share insights about the data with data consumers Y, Z, and W without disclosing the raw data. Data consumers send queries to X, where queries are processed and their responses sent back to the consumer. X has control on what queries to answer and therefore can control the confidentiality of data [12]. However, because data consumers do not have access to the raw database, they cannot verify the correctness of processing.

In this paper, we build on cryptographic solutions, namely *zero-knowledge proofs (ZKP)* [17, 21], to address the challenges of ensuring both confidentiality and provability in database query processing [14, 42, 43]. ZKP constitutes a powerful cryptographic tool that enables one party, the prover, to convince another party, the verifier, of the correctness of a statement without revealing any specific information about the statement itself. The protocols used in ZKP systems can be made interactive and non-interactive ZKP. Interactive ZKP utilize cryptographic protocols wherein two entities, namely a *prover* and a *verifier*, dynamically exchange messages to establish the validity of a statement without revealing any sensitive information. Previous research has introduced interactive ZKP for verifying SQL queries [30, 47, 49]. The rationale behind using interactive ZKP for SQL query verification (like ZKSQL [30]) is that they enable the prover to engage with the verifier in multiple rounds, incrementally constructing and verifying parts of the proof. This step-by-step interaction often results in smaller circuit sizes, as the prover can break down the computation into manageable parts rather than generating a single large, complex proof upfront. However, the interactive nature introduces challenges related to synchronization and availability between the prover and verifier. Specifically, interactive ZKP are accompanied by two main drawbacks that currently limit their widespread adoption in practical applications: (1) They necessitate interaction between the prover and verifier throughout the proof protocol, imposing requirements for availability and synchronization between the involved parties. This challenge exacerbates failures and

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SIGMOD'25, June 22-27, 2025, Berlin, Germany

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM

<https://doi.org/XXXXXXX.XXXXXXX>

PoneglyphDB: 用于任意 SQL 查询验证的高效非交互式零知识证明

顾
加州大学尔湾分校
美国尔湾
binbing@uci.edu

方俊成
美国加州大学尔湾分校, 尔湾,
美国 junchf1@uci.edu

费萨尔·纳瓦布
加州大学尔湾分校
美国尔湾
nawabf@uci.edu

抽象的

在涉及敏感数据的数据库应用中, 数据保密性和可验证性 (可证明性) 查询处理的双重需求至关重要。本文介绍了一种名为 PoneglyphDB 的数据库系统, 该系统利用非交互式零知识证明 (ZKP) 来同时支持保密性和可验证性。与传统数据库不同, PoneglyphDB 通过确保原始数据完全保留在主机端来增强保密性, 同时通过向客户端提供证明来验证查询响应的正确性。

本文的主要创新之处在于提出了一种高效的零知识证明 (ZKP) 设计 (称为电路), 用于 SQL 查询处理中的基本操作。这些基本操作电路可以组合成更大型、更复杂查询的 ZKP 电路。PoneglyphDB 的电路经过精心设计, 利用了密码学领域的最新进展, 例如基于 PLONK 的电路、递归证明组合技术以及低阶多项式约束设计, 从而实现了高效运行。我们使用标准的 TPC-H 基准测试验证了 PoneglyphDB 的性能。实验结果表明, PoneglyphDB 能够高效地兼顾保密性和可证明性, 优于现有的最先进的 ZKP 方法。

关键词

零知识证明, 可验证的 SQL 查询

ACM 参考格式:

Binbin Gu, Juncheng Fang 和 Faisal Nawab. 2025 年。PoneglyphDB: 用于任意 SQL 查询验证的高效非交互式零知识证明。tion. 在国际数据管理会议论文集 (SIGMOD'25)。ACM, 柏林, 德国, 15 页。https://doi.org/XXXXXXX。XXXXXXX

1 引言

由于数据库是各种应用程序的基石, 因此它们承担的责任不仅限于数据存储, 还包括保护敏感信息。这一点至关重要, 尤其对于存储个人数据的数据库而言。此外, 在包含人口普查数据或失业统计数据等敏感信息的应用程序中, 用户必须信任数据库所有者使用的是正确的数据库 (即与……一致的数据库)。

所声称的数据库) 并且所提供的答案准确反映了预期的计算结果。

我们致力于提供数据库解决方案, 以确保以下两个特性: (1) 保密性: 这意味着原始数据仅保存在主机 (即服务提供商) 处, 不会与其他任何节点共享或公开。其他节点仅接收其向主机节点发送的特定查询的响应, 且该查询必须经主机节点同意处理。(2) 可证明性: 确保处理的正确性。这意味着主机节点计算并返回给用户的结果反映了私有数据库上查询的正确处理。这要求主机节点向用户提供查询响应正确性的证明。

例如, 在医疗保健领域, 医学研究机构会收集患者数据用于合作研究。机构 X 可能希望与数据使用者 Y、Z 和 W 分享数据分析结果, 但又不希望泄露原始数据。数据使用者向 X 发送查询, X 处理查询并将结果返回给使用者。X 可以控制要回答哪些查询, 因此可以控制数据的保密性 [12]。然而, 由于数据使用者无法访问原始数据库, 因此他们无法验证数据处理的正确性。

本文基于密码学解决方案, 特别是零知识证明 (ZKP) [17, 21], 来解决数据库查询处理中同时保证机密性和可证明性的挑战[14, 42, 43]。ZKP 是一种强大的密码学工具, 它允许一方 (证明者) 在不泄露任何关于语句本身的具体信息的情况下, 使另一方 (验证者) 相信某个语句的正确性。ZKP 系统中使用的协议可以分为交互式和非交互式 ZKP。交互式 ZKP 利用密码学协议, 其中证明者和验证者这两个实体动态地交换消息, 以在不泄露任何敏感信息的情况下验证语句的有效性。之前的研究已经引入了交互式 ZKP 来验证 SQL 查询[30, 47, 49]。使用交互式零知识证明 (ZKP) 进行 SQL 查询验证 (例如 ZKSQL [30]) 的理由在于, 它允许证明者与验证者进行多轮交互, 逐步构建和验证证明的各个部分。这种逐步交互通常可以减小电路规模, 因为证明者可以将计算分解成易于管理的部分, 而不是预先生成一个庞大而复杂的证明。然而, 交互式特性也带来了与证明者和验证者之间的同步和可用性相关的挑战。具体而言, 交互式 ZKP 存在两个主要缺点, 目前限制了其在实际应用中的广泛采用: (1) 它要求证明者和验证者在整个证明协议过程中进行交互, 从而对参与方之间的可用性和同步提出了要求。这一挑战会加剧失败, 并且

允许出于个人或课堂使用目的, 免费复制本作品的全部或部分内容, 但前提是不得出于盈利或商业目的制作或分发副本, 且副本必须包含本声明以及首页的完整引用信息。对于本作品中非作者所有的部分, 必须尊重其版权。允许摘录并注明出处。其他任何形式的复制、再版、发布到服务器或分发到邮件列表的行为, 均需事先获得明确许可和/或支付费用。请联系 permissions@acm.org 申请许可。

SIGMOD'25, 2025 年 6 月 22 日至 27 日, 德国柏林
© 2025 版权归所有者/作者所有。出版权已授权给 ACM。ACM ISBN 978-x-xxxx-xxxx-x/YY/MM https://doi.org/XXXXXXX.XXXXXXX

	Zero-knowledge	Non-interactive	Arbitrary SQL queries
IntegriDB [49]	✗	✗	✗
vSQL [47]	✗	✗	✓
vSQL+ [48]	✓	✗	N/A
ZKSQL [30]	✓	✗	✓
PoneglyphDB	✓	✓	✓

Table 1: Comparing cryptography-based methods for verifiable SQL queries.

timeouts. (2) They lack transferability, as only the specific verifier(s) engaged in the original protocol possess the capability to verify the proof. This excludes the possibility of reusing (and caching) previously computed responses and proofs for multiple verifiers.

On the other hand, non-interactive ZKP is a cryptographic protocol class designed to establish the validity of a statement without the necessity for a dynamic exchange of messages between the prover and verifier. Unlike interactive ZKP, non-interactive ZKP empowers a prover to generate a single, self-contained proof that can be subsequently verified by any party possessing the necessary verification key. This characteristic eliminates the requirement for real-time interaction during the proof protocol, providing greater flexibility and efficiency in scenarios where asynchronous verification or limited interactivity is desired. Non-interactive ZKP, however, can experience significant overheads if the underlying cryptographic circuits are not carefully optimized.

In this paper, we propose a database system named PoneglyphDB, which incorporates non-interactive ZKP and the recursive proof composition technique. PoneglyphDB achieves both confidentiality and provability through non-interactive ZKP. PoneglyphDB differs from traditional databases by incorporating ZKP *circuits*, which are sets of equality constraints over arithmetic expressions designed to mimic the steps of query processing. These circuits enable ZKP frameworks to generate proofs of correctness for computations. In PoneglyphDB, we design circuits to represent basic query operations such as range checks, sorting, group-by, and joins, which are combined to handle more complex queries. Circuit design directly impacts proof generation performance. We optimize PoneglyphDB's circuits using the PLONKish framework [46], considering factors such as circuit size, polynomial degree, and batching construction. Additionally, PoneglyphDB leverages a recursive structure for composing proofs of multiple statements, reducing the overall proof size and computational overhead. This is made possible by advancements in proving systems that utilize recursive proof composition techniques [9, 28].

Table 1 compares PoneglyphDB and prior research on verifiable database systems [30, 47, 49] in terms of three properties: (1) the assurance of zero-knowledge (i.e., confidentiality), (2) non-interactive operability, and (3) applicability to arbitrary SQL queries. The protocols used in vSQL and vSQL+ are originally presented in an interactive manner, however they can be made non-interactive via the Fiat-Shamir heuristic [16]. To the best of our knowledge, PoneglyphDB is the first system that achieves all these desirable properties. Also, in the evaluation section, we show that PoneglyphDB performance outperforms the state-of-the-art ZKP methods.

This paper is structured as follows: Section 2 presents background material. Section 3 proposes the general design of PoneglyphDB followed by the detailed design in Section 4. Section 5 presents our experimental evaluations. Section 6 presents related work and Section 7 concludes the paper.

2 Preliminaries

2.1 Zero-Knowledge Proofs

In the area of ZKP, a prover can convincingly demonstrate the truth of a given statement to a verifier without divulging any additional information. Specifically, ZKP empowers a prover $\mathcal{P}$ with a private witness w (a “private witness” refers to a piece of secret information such as a secret input, e.g., database) to validate the truth of a public statement $\mathcal{F}$ (with respect to w) to a verifier $\mathcal{V}$, while preserving the confidentiality of the underlying information w . For instance, suppose a database contains information about employees, and the data owner wants to prove to the verifier that the average salary of the employees is a certain number without disclosing individual salary details. In this scenario, the private witness w is the information about individual salaries, and the public statement $\mathcal{F}$ is the average salary computed by the SQL query.

In scenarios with multiple verifiers, such as healthcare settings where various institutions need to verify query results, interactive protocols can become impractical due to the need for multiple rounds of communication with each verifier. Non-interactive ZKP addresses this issue by allowing the prover to generate a single proof that can be verified by all parties without further interaction. For public-coin interactive protocols [21], the Fiat-Shamir heuristic [16] effectively converts them into non-interactive ones (e.g., vSQL), maintaining efficiency and minimizing computational overhead while supporting asynchronous verification.

2.2 Arithmetization

The use of arithmetic circuits is the most common paradigm for expressing computations within ZKP systems. We introduce the PLONKish arithmetization [46] that we use in PoneglyphDB. We emphasize that the PLONKish circuits used in PoneglyphDB serve primarily as a vehicle to demonstrate the potential of non-interactive ZK proofs within database management systems (DBMSs), rather than representing a state-of-the-art protocol.

PLONKish circuits are defined in terms of a rectangular matrix of values. We refer to rows, columns, and cells of this matrix with the conventional meanings. In the following, we introduce the key definitions of PLONKish circuits that are relevant to our circuit optimization.

1. **Fixed columns.** Fixed columns are fixed by the circuit. The values in the fixed columns are usually constants.
2. **Advice columns.** Advice columns correspond to *witness* values. These are private inputs and intermediate values generated during the circuit computation.
3. **Instance columns.** Instance columns are used for any elements shared between the prover and verifier. In most cases, they are used for public inputs and outputs.
4. **Equality constraints.** Equality constraints specify that two given cells must have equal values.
5. **Polynomial constraints.** For each row in the matrix, the multivariate polynomials over the field $\mathbb{F}$ must be evaluated to zero.

Example 2.1. Figure 1 illustrates the PLONKish circuit for calculating the function $f(x, y, z) = 3 * (x + y) * z$. The circuit utilizes three advice columns—advice1, advice2, and advice3—to store the private inputs and intermediate values during computation.

In row 0, the values for x and y are put into the cells in advice1 and advice2, respectively (i.e. $x = a1$ and $y = b1$). A polynomial constraint

零知识非交互式任意 SQL 查询	IntegriDB [49]	%% % vSQL [47]	%% % ! vSQL+ [48]	! % N/A
ZKSQL [30]	! % ! PoneglyphDB !!!			

表 1: 比较基于密码学的可验证 SQL 查询方法。

超时。(2) 它们缺乏可移植性, 因为只有参与原始协议的特定验证者才有能力验证证明。这排除了为多个验证者重用 (和缓存) 先前计算的响应和证明的可能性。

另一方面, 非交互式零知识证明 (ZKP) 是一种密码协议, 旨在无需证明者和验证者之间动态交换消息即可验证声明的有效性。与交互式 ZKP 不同, 非交互式 ZKP 允许证明者生成一个独立的、自包含的证明, 任何拥有必要验证密钥的一方都可以对其进行验证。这一特性消除了证明协议过程中实时交互的需求, 从而在需要异步验证或有限交互的场景中提供了更大的灵活性和效率。然而, 如果底层密码电路没有经过精心优化, 非交互式 ZKP 可能会产生显著的开销。

本文提出了一种名为 PoneglyphDB 的数据库系统, 该系统融合了非交互式零知识证明 (ZKP) 和递归证明组合技术。PoneglyphDB 通过非交互式 ZKP 实现了保密性和可证明性。与传统数据库不同, PoneglyphDB 引入了 ZKP 电路, 这些电路是一组针对算术表达式的等式约束, 旨在模拟查询处理步骤。这些电路使得 ZKP 框架能够生成计算正确性的证明。在 PoneglyphDB 中, 我们设计了电路来表示基本查询操作, 例如范围检查、排序、分组和连接, 并将这些电路组合起来以处理更复杂的查询。电路设计直接影响证明生成性能。我们使用 PLONKish 框架 [46] 对 PoneglyphDB 的电路进行了优化, 并考虑了电路大小、多项式阶数和批处理构造等因素。此外, PoneglyphDB 利用递归结构来组合多个语句的证明, 从而减少整体证明规模和计算开销。这得益于利用递归证明组合技术的证明系统的进步 [9, 28]。

表 1 从三个方面比较了 PoneglyphDB 与先前关于可验证数据库系统的研究 [30, 47, 49]: (1) 零知识保证 (即保密性), (2) 非交互式操作性, 以及 (3) 对任意 SQL 查询的适用性。vSQL 和 vSQL+ 中使用的协议最初是以交互式方式呈现的, 但可以通过 FiatShamir 启发式算法 [16] 实现非交互式操作。据我们所知, PoneglyphDB 是第一个实现所有这些理想特性的系统。此外, 在评估部分, 我们展示了 PoneglyphDB 的性能优于目前最先进的零知识证明 (ZKP) 方法。

本文结构如下: 第 2 节介绍背景资料; 第 3 节提出 PoneglyphDB 的总体设计; 第 4 节详细阐述其设计; 第 5 节介绍实验评估结果; 第 6 节介绍相关工作; 第 7 节总结全文。

2 预赛

2.1 零知识证明

在零知识证明 (ZKP) 领域, 证明者可以在不泄露任何额外信息的情况下, 令人信服地向验证者证明给定陈述的真值。具体来说, ZKP 赋予证明者 P 一个私有证物 w (“私有证物”指的是一段秘密信息, 例如秘密输入, 比如数据库), 使其能够向验证者 V 验证公开陈述 F (关于 w) 的真值, 同时确保底层信息 w 的机密性。例如, 假设一个数据库包含员工信息, 数据所有者希望向验证者证明员工的平均工资是某个特定值, 但又不想透露具体的员工工资信息。在这种情况下, 私有证物 w 就是员工的工资信息, 而公开陈述 F 则是通过 SQL 查询计算出的平均工资。

在涉及多个验证者的场景中, 例如医疗保健领域, 多个机构需要验证查询结果, 交互式协议由于需要与每个验证者进行多轮通信而变得不切实际。非交互式零知识证明 (ZKP) 通过允许证明者生成一个所有参与方无需进一步交互即可验证的单一证明来解决这个问题。对于公币交互式协议 [21], Fiat-Shamir 启发式算法 [16] 可以有效地将其转换为非交互式协议 (例如, vSQL), 在保持效率和最小化计算开销的同时, 支持异步验证。

2.2 算术运算

在零知识证明 (ZKP) 系统中, 使用算术电路是表达计算的最常用范式。我们引入了 PLONK 式算术化方法 [46], 并在 PoneglyphDB 中使用它。我们强调, PoneglyphDB 中使用的 PLONK 式电路主要是为了展示数据库管理系统 (DBMS) 中非交互式零知识证明的潜力, 而不是代表一种最先进的协议。

PLONK 电路是用一个矩形矩阵来定义的。我们用常规含义来指代该矩阵的行、列和单元格。下面, 我们将介绍与电路优化相关的 PLONK 电路的关键定义。

1. 固定列。固定列的值由电路决定。固定列中的值通常是常数。
2. 建议列。建议列对应于见证值。这些是私有输入和电路计算过程中生成的中间值。
3. 实例列。实例列用于证明器和验证器之间共享的任何元素。在大多数情况下, 它们用于公共输入和输出。
4. 相等约束。相等约束规定两个给定单元格的值必须相等。
5. 多项式约束。对于矩阵中的每一行, 域 F 上的多元多项式的值必须为零。

例 2.1。图 1 展示了用于计算函数 $f(x,y,z) = 3 * (x + y) * z$ 的 PLONKish 电路。该电路利用三个建议列——advice1、advice2 和 advice3——来存储计算过程中的私有输入和中间值。

在第 0 行中, 将 x 和 y 的值分别放入 advice1 和 advice2 的单元格中 (即 $x=a1$ 和 $y=b1$)。多项式约束

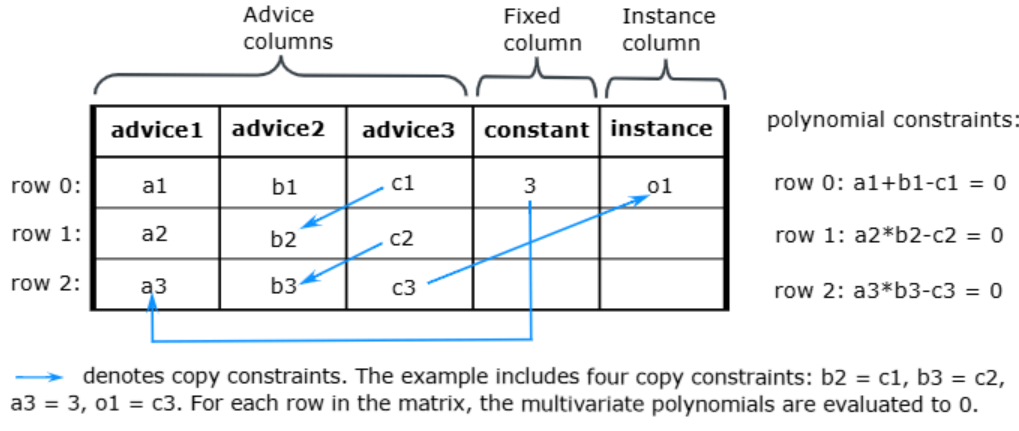


Figure 1: PLONKish circuits illustration for calculating the function $f(x,y,z)=3*(x+y)*z$.

is introduced to ensure that $c1 = a1 + b1$, calculating $x + y$. Next in row 1, the value of the input z (i.e. $z = a2$) is put into the cell in advice1. Since $a1 + b1$ was computed as $c1$, its value is propagated to $b2$ and an equality constraint sets $b2 = c1$. A multiplier gate (with polynomial constraints) then calculates $c2 = a2 * b2$, evaluating $a2 * b2 - c2 = 0$. Finally in row 2, the constant 3 is brought into advice1 at the cell of $a3$ and fixed by the copy (or equality) constraint $a3 = 3$. The previous intermediate result $(a1 + b1) * a2 = c2$ is copied into $b3$, with an equality constraint $b3 = c2$. The final output cell $c3$ calculates $a3 * b3$, which evaluates to $3 * (a1 + b1) * a2$.

This result is copied into the public instance column at $o1$, allowing the verifier to read off the final output. Through a series of advice columns, equality constraints, and polynomial operations, the circuit computes the desired function in a modular fashion while keeping intermediate values private. The verifier only sees the final outputs revealed in the instance column.

To facilitate the modularity of PLONKish circuits, the designs of basic functions are represented as *gates*. A gate is a collection of columns and constraints that together implement a basic operation, such as division and multiplication. In our case, we will define gates for basic operations for query processing such as range checks and sorting. Gates can then be combined to implement more complex functions (or more complex gates).

3 System Overview

3.1 System Model

In PoneglyphDB, the prover $\mathcal{P}$ hosts a copy of a private database $\mathcal{DB}$. The prover receives queries from one or more verifiers, collectively denoted $\mathcal{V}$. The prover does not provide raw access to the data in the database. However, it answers queries sent by the verifiers.

3.2 Workflow Overview

PoneglyphDB operates in five key phases, as illustrated in Figure 2:

(1) Sending SQL Queries: The client, who will eventually assume the role of a verifier, sends SQL queries for execution against a private database, directly to the prover. The prover, holding exclusive access to this private database, is tasked with executing the queries.

(2) Circuit Construction: Upon receipt of the query request, the prover is then responsible for constructing the SQL queries and database commitment into arithmetic circuits. These circuits, delineated by gates and polynomial constraints, performs the computations that need to be proven. The SQL circuit encodes the desired SQL logic to be evaluated. This circuit allows for the computation of different inputs provided by the verifiers during the verification phase.

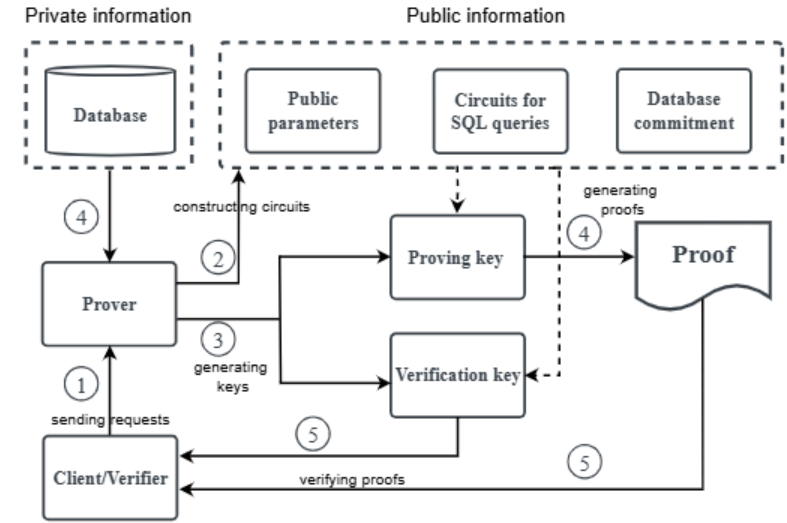


Figure 2: PoneglyphDB Workflow.

A database commitment is a cryptographic representation of a database state, enabling proof of properties about the data without revealing the data itself. This allows the prover to include evidence in the proof that the query was indeed processed on $\mathcal{DB}$. We employ the Inner Product Argument (IPA) protocol [8], operating over a 254-bit prime field, to generate this commitment. We choose the IPA protocol for the following reasons: (1) the proving time is typically linear with respect to the circuit size or the degree of the committed polynomial, (2) the proof size and verification time are logarithmic in the circuit size, due to the recursive structure of the inner product proof [9], and (3) it is compatible with PLONKish-based circuit designs.

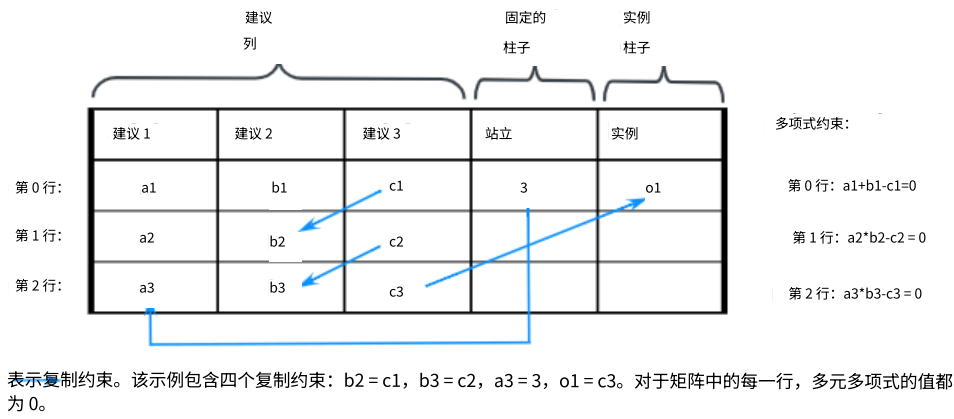
By leveraging IPA, we can encode both the database commitment and the desired SQL logic within a unified framework [8, 10]. The public parameters are a shared foundation for both provers and verifiers in proof creation and validation. This process utilizes publicly verifiable randomness and avoids the need for a trusted setup [13]. Initial proof creation uses only public information, forming the basis for subsequent proofs [9].

(3) Key Generation: Leveraging the public parameters and the circuit description, PoneglyphDB generates a proving key. This key is used to generate proofs corresponding to the circuit. Concurrently, a verification key is also produced, grounded in the same public parameters and circuit description. This verification key empowers verifiers to authenticate the proofs associated with this circuit.

(4) Proof Generation: The prover uses the proving key to generate a ZKP validating the correct computation of the SQL query over the private database. Leveraging the previously established database commitment, the prover creates a verifiable link between the committed database, query execution, and result. This process incorporates commitments to relevant database rows and intermediate computation steps, forming a chain of verifiable commitments from the initial database state to the final query output.

(5) Proof Verification: Finally, the verifier employs the public verification key to efficiently validate the ZKP received from the prover. The verification algorithm essentially interpolates the wiring polynomials and checks that all constraints are satisfied.

PoneglyphDB Positioning in the Workflow. In PoneglyphDB, we utilize existing work from the cryptography community to perform the steps of key and proof generation and verification. The main contribution of our work is in the design of SQL circuits that allow for efficient proving and verification given the utilized framework [44].

图 1: PLONKish 电路图，用于计算函数 $f(x, y, z) = 3 * (x + y) * z$ 。

引入此约束是为了确保 $c1 = a1 + b1$ ，从而计算 $x + y$ 。接下来，在第 1 行中，输入 z 的值（即 $z = a2$ ）被放入 advice1 的单元格中。由于 $a1 + b1$ 已计算为 $c1$ ，因此其值会传递到 $b2$ ，并且等式约束会设置 $b2 = c1$ 。然后，乘法器门（带有多项式约束）计算 $c2 = a2 * b2$ ，并计算 $a2 * b2 - c2 = 0$ 。最后，在第 2 行，常数 3 被引入到 $a3$ 单元格的 advice1 中，并通过复制（或等式）约束 $a3 = 3$ 固定。之前的中间结果 $(a1 + b1) * a2 = c2$ 被复制到 $b3$ 中，并带有等式约束 $b3 = c2$ 。最终输出单元格 $c3$ 计算 $a3 * b3$ ，其值为 $3 * (a1 + b1) * a2$ 。

该结果被复制到 o1 处的公共实例列中，以便验证者读取最终输出。通过一系列建议列、等式约束和多项式运算，电路以模块化的方式计算所需函数，同时将中间值保密。验证者只能看到实例列中显示的最终输出。

为了便于 PLONK 电路的模块化设计，基本功能的设计以门电路的形式呈现。门电路是一组列和约束的集合，共同实现一个基本运算，例如除法和乘法。在本例中，我们将定义用于查询处理的基本运算的门电路，例如范围检查和排序。然后，可以将这些门电路组合起来，实现更复杂的功能（或更复杂的门电路）。

3 系统概述

3.1 系统模型

在 PoneglyphDB 中，证明者 P 维护着一个私有数据库 DB 的副本。证明者接收来自一个或多个验证者（统称为 V）的查询。证明者不提供对数据库中数据的原始访问权限，但它会回答验证者发送的查询。

3.2 工作流程概述

如图 2 所示，PoneglyphDB 的运行分为五个关键阶段：(1) 发送 SQL 查询：客户端（最终将扮演验证者的角色）直接向证明者发送针对私有数据库的 SQL 查询。证明者拥有对该私有数据库的独占访问权限，负责执行这些查询。

(2) 电路构建：收到查询请求后，证明者负责将 SQL 查询和数据库提交构建成算术电路。这些电路由门电路和多项式约束构成，执行需要证明的计算。SQL 电路编码了待评估的 SQL 逻辑。该电路允许计算验证者在验证阶段提供的不同输入。

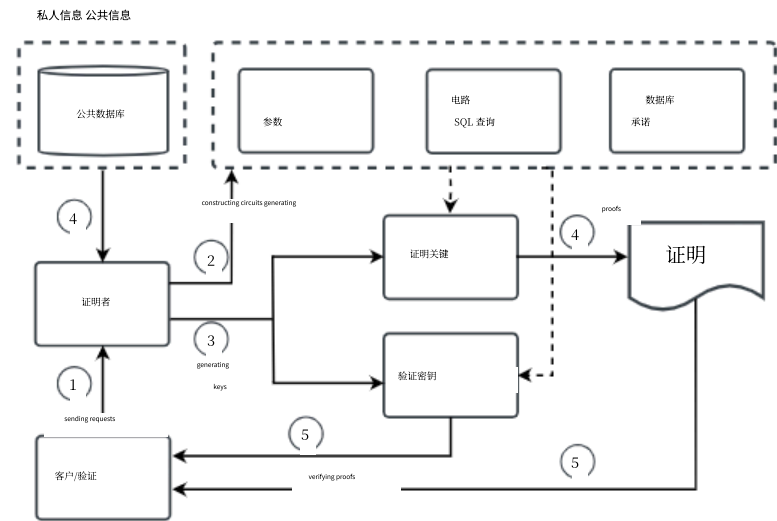


图 2: PoneglyphDB 工作流程。

数据库承诺是数据库状态的一种加密表示，它允许在不泄露数据本身的情况下证明数据的属性。这使得证明者能够在证明中包含查询确实在数据库上处理的证据。我们采用基于 254 位素数域的内积论证 (IPA) 协议 [8] 来生成此承诺。我们选择 IPA 协议的原因如下：(1) 证明时间通常与电路规模或承诺多项式的次数呈线性关系；(2) 由于内积证明的递归结构 [9]，证明规模和验证时间与电路规模呈对数关系；(3) 它与基于 PLONK 的电路设计兼容。

通过利用 IPA，我们可以将数据库承诺和所需的 SQL 逻辑编码到一个统一的框架中 [8, 10]。公共参数为证明者和验证者在证明创建和验证过程中提供了共享的基础。该过程利用了可公开验证的随机性，避免了对可信设置的依赖 [13]。初始证明创建仅使用公共信息，为后续证明奠定了基础 [9]。

(3) 密钥生成：PoneglyphDB 利用公开参数和电路描述生成一个证明密钥。该密钥用于生成与电路对应的证明。同时，基于相同的公开参数和电路描述，还会生成一个验证密钥。该验证密钥使验证者能够验证与该电路关联的证明。

(4) 证明生成：证明者使用证明密钥生成零知识证明 (ZKP)，以验证对私有数据库执行的 SQL 查询的正确性。利用先前建立的数据库承诺，证明者在已承诺的数据库、查询执行和结果之间建立可验证的链接。此过程包含对相关数据库行和中间计算步骤的承诺，从而形成从初始数据库状态到最终查询输出的一系列可验证承诺。

(5) 证明验证：最后，验证者使用公钥高效地验证从证明者处收到的零知识证明。验证算法本质上是对连接多项式进行插值，并检查所有约束是否都得到满足。

PoneglyphDB 在工作流程中的定位。在 PoneglyphDB 中，我们利用密码学界现有的工作成果来完成密钥和证明的生成和验证步骤。我们工作的主要贡献在于设计了 SQL 电路，使得在所采用的框架下能够高效地进行证明和验证 [44]。

3.3 Application Framework Disucssion

Trust Model. In the PoneglyphDB framework, the core parties involved are the database owner, acting as the prover, and the clients, acting as the verifiers. The prover and verifiers do not trust each other as we detail below. There is an additional entity, the auditor, that both the prover and verifier trust (there can be multiple auditors). The following are trust concerns between the parties:

1. **The prover may use a fake database:** The client does not trust the prover to be using the correct database to run the query. For example, a server responsible for running SQL queries might use a tampered or fake database to process a query and provide incorrect results to the verifier.
2. **The prover may process the query incorrectly:** Even if the correct database is used, the client cannot trust the accuracy of the results returned by the prover. The server could mistakenly or intentionally alter or fabricate the query results.
3. **The verifier may attempt to leak data:** On the other hand, the prover fears that the client might attempt to extract or infer additional information about the underlying database beyond what is allowed by the query. The client could craft a series of queries to uncover sensitive patterns, private data, or proprietary information, breaching the prover's confidentiality.

We now map this trust model to a real-world scenario. In general, this trust model applies to cases when a database owner has sensitive data and wants to enable other entities (clients) to query the database without revealing data beyond what the database owner allows. In the healthcare scenario, a hospital H wants to provide query access to its database of patient data. It does not want to reveal all data, but would allow answering specific types of queries. The clients can be healthcare providers or research institutions that want to utilize H 's database for their research. In this scenario, an auditor can be a government or regulatory entity that both H and clients trust. Next, we map the trust model assumptions onto this example:

1. The clients do not trust the hospital H to be using a correct database of patient's data: The hospital H does not reveal the raw database to the clients, and therefore clients cannot attest to the authenticity of the database used to answer queries.
2. The clients do not trust the hospital H to process queries on the database correctly: The hospital H may process the queries on the database incorrectly (e.g., by using approximation techniques to save costs, or by completely fabricating results).
3. The clients may attempt to leak data from query responses: The clients may want to know more information about the database of H than what is provided in the answer to the query.

To address these trust issues, PoneglyphDB introduces the following measures:

(1) Cryptographic Commitment to the Database: PoneglyphDB requires the prover to make an irrevocable cryptographic commitment to the database. This ensures that the prover cannot substitute the real database with a bogus one while still producing valid query results. The commitment is made public and shared in an irrevocable and immutable manner, e.g., by utilizing a decentralized blockchain such as Ethereum. This ensures that the prover will use the same database that corresponds to the database commitment as clients have access to an irrevocable, immutable database commitment that they can compare with the commitment used in the received proof. The

database commitment can also be audited by a third-party auditor (e.g., a regulatory or government entity trusted by both the prover and verifiers). The auditor in this case reads the raw database from the database owner, verifies its authenticity, and validates that the database commitment of the authentic database corresponds to the commitment that is shared in the blockchain and accessible by the verifiers.

To prove the correctness of database updates and generate new commitments, a naive method would be to recompute the commitment for the entire database after each update. However, this approach is inefficient for large databases. A more advanced method might involve using a Merkle tree structure [32], where only the affected subtree is updated and recomputed, enabling efficient localized proof generation. Another potential method involves leveraging batch update techniques, where multiple updates are aggregated into a single proof.

(2) Ensuring Query Result Correctness: PoneglyphDB employs a constraint system that encodes SQL queries as circuits, ensuring that the verifier can confirm the query results are derived from a pre-defined and correct computation process. This process ensures the prover cannot return fabricated results.

(3) Preserving Data Privacy: To ensure the owner's database privacy, PoneglyphDB employs ZKP. These proofs guarantee that the client only receives the query result without extracting or inferring any additional information about the underlying database. The zero-knowledge property protects the database from unwanted data leakage beyond what is revealed from the query response. This zero-knowledge property can be combined with other techniques that prevent other types of leakage of private data. For this reason, it is important to distinguish between the types of leakage that are prevented or allowed by the zero-knowledge property and complement it with other techniques. A ZKP does not reveal information beyond the query response, but it does reveal what is part of the query response (and anything that can be implied by the query response). Therefore, if there is data that a user should not have access to but is part of the query response, then it is leaked to the client. To prevent this type of leakage, ZKP can be combined with techniques such as access control, data masking, query filtering, and policy management to prevent processing queries on data that the user would not have access to [1]. These techniques, typically applied as pre-processing steps, allow the prover to decide whether a query should be processed. For example, if the client's query asks to get raw data that should not be revealed, then the prover would not process the query. Query filtering, in particular, helps prevent leakage by modifying the query plan to exclude sensitive information before execution, ensuring that only authorized data is included in the response. Another type of leakage is to infer information about the database that were not intended to be revealed directly in the query response. This type of leakage can be prevented by incorporating differential privacy techniques as we discuss at the end of Section 3.4.

With these techniques, and reflecting back to the mapping to the real-world scenario above, we observe the following: (1) PoneglyphDB ensures that the hospital H uses an authentic database to process queries. This is by utilizing a trusted third-party auditor that verifies that the publicly shared irrevocable and immutable database commitment corresponds to an authentic database. If a prover attempts to utilize a different database, then the client is able to discover this as the client would match the publicly shared database commitment that is authenticated by an auditor with the database commitment that is used

3.3 应用框架讨论

信任模型。在 PoneglyphDB 框架中，核心参与方包括数据库所有者（作为证明者）和客户端（作为验证者）。正如我们将在下文详述的，证明者和验证者之间互不信任。此外，还有一个由证明者和验证者共同信任的实体——审计员（可以有多个审计员）。各方之间的信任问题如下：1. 证明者可能使用伪造的数据库：客户端不信任证明者使用正确的数据库来运行查询。例如，负责运行 SQL 查询的服务器可能使用篡改过的或伪造的数据库来处理查询，并向验证者提供错误的结果。

- 2. 证明器可能会错误地处理查询：即使正确 - 如果使用 rect 数据库，客户端无法信任验证器返回结果的准确性。服务器可能会错误地或故意地篡改或伪造查询结果。
- 3. 验证者可能会试图泄露数据：另一方面，验证者担心客户可能会试图提取或推断出查询允许范围之外的底层数据库信息。客户可能会精心设计一系列查询，以发现敏感模式、私人数据或专有信息，从而违反验证者的保密义务。

现在，我们将此信任模型映射到一个实际场景。一般来说，此信任模型适用于数据库所有者拥有敏感数据，并希望允许其他实体（客户）查询数据库，但又不希望泄露超出数据库所有者权限范围的数据的情况。在医疗保健场景中，医院 H 希望提供对其患者数据库的查询访问权限。它不希望泄露所有数据，但允许回答特定类型的查询。客户可以是希望利用 H 的数据库进行研究的医疗服务提供者或研究机构。在这种情况下，审计人员可以是 H 和客户都信任的政府或监管机构。接下来，我们将信任模型的假设映射到此示例中：

- 1. 客户不信任医院 H 使用正确的病人数据数据库：医院 H 不向客户公开原始数据库，因此客户无法证明用于回答查询的数据库的真实性。
- 2. 客户不信任医院 H 能够正确处理数据库查询：医院 H 可能会错误地处理数据库查询（例如，使用近似技术来节省成本，或者完全捏造结果）。
- 3. 客户端可能会试图从查询响应中泄露数据：客户端可能想要了解有关 H 数据库的更多信息，而不是查询答案中提供的信息。

为了解决这些信任问题，PoneglyphDB 采取了以下措施：

- (1) 对数据库的加密承诺：PoneglyphDB 证明者需要对数据库做出不可撤销的加密承诺。这确保了证明者无法在生成有效查询结果的同时，用伪造的数据库替换真实的数据库。该承诺以不可撤销且不可篡改的方式公开共享，例如，通过以太坊等去中心化区块链。这确保了证明者将使用与数据库承诺相对应的数据库，因为客户端可以访问不可撤销且不可篡改的数据库承诺，并将其与接收到的证明中使用的承诺进行比较。

数据库承诺也可以由第三方审计机构（例如，证明者和验证者都信任的监管机构或政府实体）进行审计。在这种情况下，审计机构会从数据库所有者处读取原始数据库，验证其真实性，并确认真实数据库的承诺与区块链中共享且验证者可以访问的承诺相符。

为了验证数据库更新的正确性并生成新的承诺，一种简单的方法是在每次更新后重新计算整个数据库的承诺。然而，这种方法对于大型数据库效率低下。一种更高级的方法可能涉及使用默克尔树结构[32]，其中仅更新和重新计算受影响的子树，从而实现高效的局部证明生成。另一种潜在方法是利用批量更新技术，将多次更新聚合到一个证明中。

- (2) 确保查询结果的正确性：PoneglyphDB 采用该约束系统将 SQL 查询编码为电路，确保验证器能够确认查询结果源自预定义的正确计算过程。此过程确保证明器无法返回伪造的结果。
- (3) 保护数据隐私：为了确保所有者的数据库隐私，PoneglyphDB 采用了零知识证明 (ZKP)。这些证明保证客户端仅接收查询结果，而不会提取或推断任何关于底层数据库的额外信息。零知识属性保护数据库免受查询响应之外的意外数据泄露。零知识属性可以与其他防止其他类型私有数据泄露的技术结合使用。因此，区分零知识属性能够防止或允许的泄露类型，并与其他技术进行补充至关重要。零知识证明不会泄露查询响应之外的信息，但它会泄露查询响应中包含的内容（以及任何可以从查询响应中推断出的信息）。因此，如果存在用户不应访问但包含在查询响应中的数据，则这些数据会被泄露给客户端。为了防止此类泄露，零知识证明 (ZKP) 可以与访问控制、数据脱敏、查询过滤和策略管理等技术相结合，以防止处理用户无权访问的数据查询[1]。这些技术通常作为预处理步骤应用，使证明者能够决定是否处理查询。例如，如果客户端的查询请求获取不应公开的原始数据，则证明者不会处理该查询。特别是查询过滤，它通过在执行前修改查询计划以排除敏感信息来帮助防止泄露，确保响应中仅包含授权数据。另一种泄露类型是推断出数据库中原本不应在查询响应中直接透露的信息。这种泄露可以通过引入差分隐私技术来防止，我们将在 3.4 节末尾讨论这些技术。

运用这些技术，并结合上述与实际场景的映射，我们可以观察到以下几点：(1) PoneglyphDB 确保医院 H 使用真实数据库来处理查询。这是通过利用可信的第三方审计机构来实现的，该机构会验证公开共享的不可撤销且不可变的数据库承诺是否与真实数据库相符。如果验证者试图使用不同的数据库，客户端就能发现这一点，因为客户端会将经审计机构验证的公开共享数据库承诺与实际使用的数据库承诺进行比对。

to process the query and received as part of the response. (2) PoneglyphDB ensures that hospital H processes the query correctly (according to the logic of the SQL query) by the ZKP constructions. (3) PoneglyphDB ensures that no information leakage occurs beyond what is revealed by the responses of hospital H. Hospital H can integrate further techniques—such as differential privacy—to ensure that other types of data leakage would not occur (as we describe in Section 3.4).

3.4 Security Model

PoneglyphDB leverages the Halo2 proving system [44], which incorporates well-established cryptographic properties such as completeness, soundness, knowledge soundness, and zero-knowledge.

- **Completeness.** If the prover can generate the PLONKish circuit (including the output) of a query, it can always convince the verifier that the PLONKish circuit of the query is true.
- **Soundness.** For any false PLONKish circuit (including any wrong witness, inputs and output), the probability of a dishonest prover successfully convincing an honest verifier is negligible.
- **Knowledge Soundness.** When the verifier is convinced the PLONKish circuit is correct, the prover actually possesses a valid witness.
- **Zero Knowledge.** The verifier learns only the information that can be inferred from the structure of the PLONKish circuit and the output of the query. No additional knowledge about the private witnesses or the database is revealed.

Figure 3 illustrates the detailed components involved in generating ZKP within PoneglyphDB. The system comprises two main parts: the PLONKish circuit and the Halo2 proving system. The PLONKish circuit serves as the input to the Halo2 proving system, which then generates ZKP for the circuit. Our primary contribution lies in the design of the PLONKish circuits, which are tailored to optimize the performance and efficiency of the ZKP generation process.

The PLONKish circuit represents the computation of a SQL query. This circuit is a mathematical representation of the logical operations and constraints involved in the SQL query execution. The prover, who possesses the private database, assigns values to all circuit variables based on the actual data from the database. This step involves mapping the data inputs to the corresponding variables in the circuit, ensuring that the computation is correctly set up for proof generation. The prover must use the database agreed upon with the verifier by utilizing the previously established database commitment.

The *Polynomial representation* component, provided by the Halo2 proving system, translates the circuit into a polynomial form. This component encodes the computation and its constraints as polynomial equations, making them suitable for ZKP.

The *Polynomial commitment* component, also provided by the Halo2 proving system, allows the prover to commit to the polynomial evaluations without revealing the actual polynomials. This ensures that the prover cannot alter the polynomials after the commitment, maintaining the integrity of the proof.

The *Halo2 proving system* takes the committed polynomials and generates opening proofs. These proofs are designed to show that the committed polynomials satisfy the polynomial constraints derived from the PLONKish circuit.

It is important to note that the PLONKish circuits implemented in PoneglyphDB are primarily intended to illustrate the feasibility of

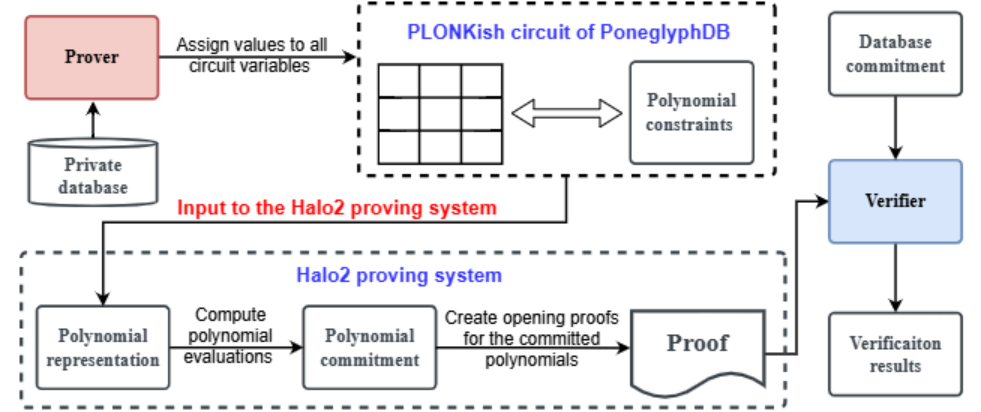


Figure 3: Detailed components for generating ZK proofs.

using non-interactive ZK proofs within DBMSs, rather than to claim they represent the latest advancements in ZK protocol design.

Guarantees. The correctness and security of PoneglyphDB depend on PLONKish circuits and the Halo2 proving system. We demonstrate the correctness of these circuits for SQL queries in Section 4.

Regarding the security of the Halo2 proving system used in PoneglyphDB, the prover and verifier engage in a non-interactive ZKP protocol utilizing Halo2’s polynomial commitment scheme and recursive proof composition. The detailed security analysis of this protocol has been rigorously established in [9, 45]. We present a high-level summary of the correctness of the system below. To analyze the cryptographic protocols employed in Halo2, we utilize the Algebraic Group Model (AGM) [18]. The AGM is used to analyze protocols that rely on discrete logarithm assumptions in prime-order groups, a fundamental aspect of Halo2’s design. The AGM evaluates the security of cryptographic protocols by requiring that adversaries must explicitly compute group elements from previously observed elements, emphasizing discrete logarithm-type assumptions in prime-order groups. PoneglyphDB guarantees completeness, soundness, knowledge soundness, and zero-knowledge properties under the AGM, assuming that all parties, including potential adversaries, are computationally bounded to probabilistic polynomial-time (PPT) algorithms. PoneglyphDB ensures that for any PPT adversary, there exists a PPT simulator such that, for any environment with arbitrary auxiliary input, the output distribution of the environment in a real-world execution (where a prover interacts with a verifier) is computationally indistinguishable from the output distribution in an ideal-world execution (where a simulator interacts with the verifier).

Oblivious circuits. The proving logic of the circuits in PoneglyphDB is designed to be oblivious. This means that the execution of the proving algorithm does not depend on the specific values of the private inputs (the witness). Instead, the prover performs the same operations regardless of the actual witness values, which helps ensure that no information about the private inputs is leaked through the proof generation process or the resulting proof itself. For example, when implementing a sorting algorithm, the circuit would compare and swap elements in a fixed pattern regardless of their actual values, ensuring the same operations are performed for any input. Similarly, for conditional statements, both branches of the condition are typically evaluated, and their results are combined using a selector value, rather than following only one branch based on the private condition. To protect the privacy of table cardinalities and intermediate result sizes in join operations, we adopt the method introduced in ZKSQL [30]. This approach involves introducing dummy tuples into our query evaluation process, effectively obscuring true data sizes while maintaining consistent row counts throughout query execution.

(2) PoneglyphDB 通过零知识证明（ZKP）构造确保医院 H 正确处理查询（根据 SQL 查询的逻辑）。(3) PoneglyphDB 确保除医院 H 的响应所揭示的信息外，不会发生其他信息泄露。医院 H 可以集成其他技术（例如差分隐私）来确保不会发生其他类型的数据泄露（如第 3.4 节所述）。

3.4 安全模型

PoneglyphDB 利用 Halo2 证明系统 [44]，该系统融合了完善的密码学特性，如完备性、可靠性、知识可靠性和零知识。

- 完备性。如果证明者能够生成查询的 PLONK 电路（包括输出），则它总能说服验证者该查询的 PLONK 电路为真。
- 可靠性。对于任何错误的 PLONKish 电路（包括任何错误的证明、输入和输出），不诚实的证明者成功说服诚实的验证者的概率可以忽略不计。
- 知识可靠性。当验证者确信 PLONKish 电路是正确的时，证明者实际上就拥有了有效的证据。
- 零知识证明。验证者仅能从 PLONKish 电路的结构和查询的输出中推断出信息。不会泄露关于私有见证人或数据库的任何额外信息。

图 3 展示了 PoneglyphDB 中生成零知识证明 (ZKP) 的详细组件。该系统包含两个主要部分：PLONKish 电路和 Halo2 证明系统。PLONKish 电路作为 Halo2 证明系统的输入，后者随后为该电路生成 ZKP。我们的主要贡献在于 PLONKish 电路的设计，该电路经过优化，旨在提升 ZKP 生成过程的性能和效率。PLONKish 电路代表 SQL 查询的计算过程。该电路以数学形式表示 SQL 查询执行过程中涉及的逻辑运算和约束。拥有私有数据库的证明者会根据数据库中的实际数据为所有电路变量赋值。此步骤涉及将数据输入映射到电路中的相应变量，从而确保计算设置正确，以便生成证明。证明者必须使用之前与验证者约定的数据库，并利用先前建立的数据库承诺。

Halo2 证明系统提供的多项式表示组件将电路转换为多项式形式。该组件将计算及其约束编码为多项式方程，使其适用于零知识证明 (ZKP)。

Halo2 证明系统提供的多项式承诺组件允许证明者承诺多项式的计算结果，而无需透露实际的多项式。这确保了证明者在承诺后无法更改多项式，从而维护了证明的完整性。

Halo2 证明系统接收已提交的多项式，并生成开式证明。这些证明旨在表明已提交的多项式满足由 PLONKish 电路导出的多项式约束。值得注意的是，PoneglyphDB 中实现的 PLONKish 电路主要目的是为了说明其可行性。

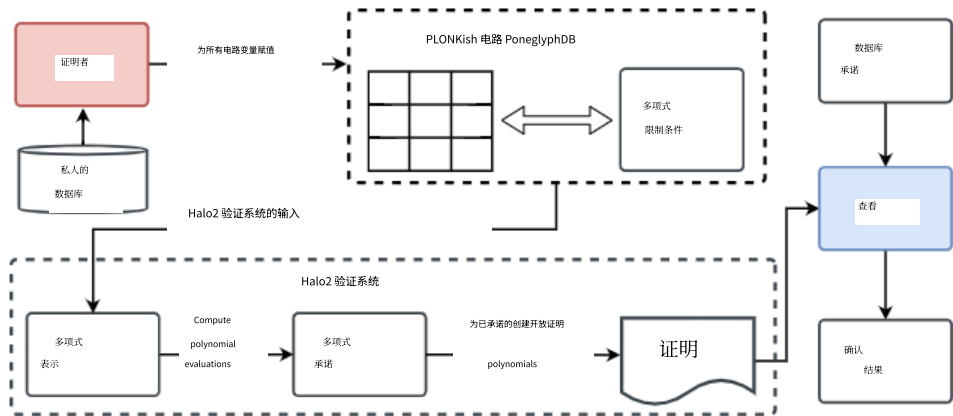


图 3：生成 ZK 证明的详细组件。

在数据库管理系统中使用非交互式零知识证明，而不是声称它们代表了零知识协议设计的最新进展。

保证。PoneglyphDB 的正确性和安全性依赖于 PLONK 电路和 Halo2 证明系统。我们在第 4 节中证明了这些电路对 SQL 查询的正确性。关于 PoneglyphDB 中使用的 Halo2 证明系统的安全性，证明者和验证者采用非交互式零知识证明（ZKP）协议，该协议利用了 Halo2 的多项式承诺方案和递归证明组合。该协议的详细安全性分析已在[9, 45]中得到严格阐述。下文将对系统的正确性进行概述。为了分析 Halo2 中使用的密码协议，我们采用了代数群模型（AGM）[18]。AGM 用于分析依赖于素数阶群中离散对数假设的协议，这是 Halo2 设计的一个基本方面。AGM 通过要求攻击者必须显式地从先前观察到的元素计算群元素来评估密码协议的安全性，从而强调素数阶群中的离散对数型假设。PoneglyphDB 在 AGM 假设下保证完备性、可靠性、知识可靠性和零知识属性，前提是所有参与方（包括潜在对手）的计算能力都受限於概率多项式时间 (PPT) 算法。PoneglyphDB 确保对于任何 PPT 对手，都存在一个 PPT 模拟器，使得对于任何具有任意辅助输入的环境，在真实世界执行（证明者与验证者交互）中该环境的输出分布与在理想世界执行（模拟器与验证者交互）中该环境的输出分布在计算上无法区分。

不经意电路。PoneglyphDB 中电路的证明逻辑被设计为不经意的。这意味着证明算法的执行不依赖于私有输入（见证）的具体值。相反，证明器执行的操作与实际的见证值无关，这有助于确保在证明生成过程或最终证明本身中不会泄露任何关于私有输入的信息。例如，在实现排序算法时，电路会按照固定的模式比较和交换元素，而不管它们的实际值如何，从而确保对任何输入都执行相同的操作。类似地，对于条件语句，通常会评估条件的两个分支，并使用选择器值将它们的结果合并，而不是仅根据私有条件执行一个分支。为了保护连接操作中表基数和中间结果大小的隐私，我们采用了 ZKSQL [30] 中引入的方法。这种方法涉及在查询评估过程中引入虚拟元组，从而有效地模糊真实的数据大小，同时在整个查询执行过程中保持行数的一致性。

3.5 Data Privacy Issues and Limitations

PoneglyphDB sends the query results to the clients. Since the proof inherently includes the result of the query, a client could, for example, issue a query like "SELECT * FROM T" to attempt to retrieve all values from T, which threatens data privacy. Beyond the direct data exposure of returned results, sensitive data can also be exposed indirectly. Specifically, when data points or records in the database are correlated, query results may still leak sensitive information about records that are not part of the response. For instance, correlation among records—such as similar attributes shared across groups or statistical dependencies—can allow a client to infer additional, sensitive details beyond the returned query results. In this way, even without direct access to records that are not included in query results, clients can potentially exploit patterns in the query response to gain insights into the broader dataset indirectly. This poses a data privacy risk that needs to be handled carefully.

To address this issue, differential privacy techniques [15, 26] could be employed to ensure that individual data points are not revealed either directly or indirectly. In this work, we do not incorporate differential privacy and leave it as an avenue of future work. We note, however, that methods of incorporating differential privacy to ZKP systems such as PoneglyphDB would lead to additional overhead in the circuit design.

4 Custom Gates

In this section, we present customized gates to represent SQL queries arithmetization in circuits. Our goal is to introduce efficient designs by creating gates that have low-degree polynomials and a smaller number of circuit constraints. We design with low-degree polynomials because ZKP relies on cryptographic primitives where evaluating higher-degree polynomials is computationally expensive.

4.1 Range Check

We first introduce a range check gate as it is involved in many SQL operations like “filter”, “sort”, “group by” and “join”.

Consider the range check statement $x \leq t$, where x is a private value and t is the public query input. A naive encoding compares x against each possible value using the polynomial equation: $\prod_{i=0}^t (x - i) = 0$. The degree of this polynomial grows linearly with t , making proof generation and verification computationally infeasible for large t . In this work, we leverage a *lookup table* [19] circuit structure to design the range check gate. The intuition behind using lookup tables lies in the idea of precomputing and storing results for a range of possible inputs. Our work builds upon the widely used Plookup framework [19], which is well-established in the ZKP domain. While there are alternatives, such as Jolt [36], we leave exploration of these methods for future work. Instead of reinventing cryptographic primitives, we focus on solving concrete implementation challenges to enhance efficiency of SQL operations within the PLONK framework. While previous works [30, 47] implement operations like filter, sort, and join using boolean or logic gate frameworks, our approach utilizes arithmetic-based PLONKish circuits. This fundamental difference introduces significant challenges, as arithmetic circuits require different optimization strategies compared to boolean circuits. Adapting SQL operators in this context necessitates a rethinking of their expression and optimization to achieve maximum efficiency.

Definition. The input to a range check gate, which checks if elements are between y_1 and y_m , consists of a column C_{in} containing n elements $\{x_1, x_2, \dots, x_n\}$ to be range-checked, and a column (lookup table) T containing m sorted values $\{y_1, y_2, \dots, y_m\}$ representing the valid range, where $y_1 < y_2 < \dots < y_m$. The output of the range check gate is a column C_{out} containing n elements $\{z_1, z_2, \dots, z_n\}$, where each z_i indicates whether x_i is within the range defined by the lookup table T . Specifically, $z_i = 1$ if $x_i \in \{y_1, y_2, \dots, y_m\}$, and $z_i = 0$ otherwise.

Design A: A single range check. We start with a simple case where we only want to prove that a single value x is in a specific range $[0, t]$, hence proving that $0 \leq x \leq t$. In the first step of constructing the circuit, we create a private array P (stored in an advice column) with the same length of set Q where the first element in P is x and the other elements are any values copied from Q (these values can be duplicates). Then, the prover supplies a permutation of P , denoted by P' where P' is private and stored in an advice column in the circuit. The values in P' are sorted so that duplicate values are row-adjacent to each other.

In the second step, we establish a fixed column to store the set Q , arranging the values in ascending order. This organization ensures that both the prover and the verifier are aware of the values and their corresponding indices within the table. Such knowledge is crucial for determining the size of the lookup table needed.

The prover supplies a permutation of Q , denoted by Q' , where Q' is private and stored in an advice column in the PLONKish circuit. The purpose of the permutation Q' is to hide the position of values in Q as the verifier knows all the information about Q . Then, the circuit can compare values in Q' and P' to check whether x (in P') is equal to some value in Q' , without revealing which value is that in Q' .

Now, we show how we can check whether the value x in P' is equal to a value in Q' . The values in Q' are arranged in a specific order such that either $P'_i = Q'_i$ or that $P'_i = P'_{i-1}$ where P'_i and Q'_i represent the i -th elements of P' and Q' respectively. Specifically, we enforce that the first values of P and Q are equal, i.e., $P'_i = Q'_i$ with $i=0$. If $P'_i \neq Q'_i$ for $i > 0$, we enforce that $P'_i = P'_{i-1}$ meaning that P'_i must be a duplicate of P'_{i-1} in this case. Therefore, these constraints guarantee that every value in P'_i is equal to some value in Q' . Formally, we enforce that either $P'_i = Q'_i$ or that $P'_i = P'_{i-1}$, using the rule:

$$0 = \begin{cases} (P'_i - Q'_i) \cdot (P'_i - P'_{i-1}) & \text{if } 1 \leq i \leq \text{len}(Q') - 1, \\ P'_i - Q'_i & \text{if } i = 0. \end{cases} \quad (1)$$

With the above polynomial constraints, the verifier knows that all the values of P' are in Q' without knowing the position information of the values in P' and Q' (therefore it does not know the exact values at each position in P' , preserving the privacy of the x value).

Since P' and Q' are permutations of P and Q , the polynomial constraints above ensure that all the values of P (and P') are in Q . To ensure this property, we develop polynomial constraints to ensure that both P' is a permutation of P and Q' is a permutation of Q :

$$\prod_{i=0}^{\text{len}(Q)-1} (P_i + \alpha)(Q_i + \beta) = \prod_{i=0}^{\text{len}(Q)-1} (P'_i + \alpha)(Q'_i + \beta) \quad (2)$$

where P_i , Q_i , P'_i and Q'_i represent the i -th element in P , Q , P' , and Q' respectively, and α and β are randomly chosen parameters. The random values α and β conceal the contents of the columns, ensuring confidentiality during verification and preventing zero products from

3.5 数据隐私问题和局限性

PoneglyphDB 会将查询结果发送给客户端。由于证明本身就包含查询结果，客户端可能会发出类似“SELECT * FROM T”这样的查询，试图从 T 中检索所有值，这会威胁数据隐私。除了返回结果的直接数据泄露之外，敏感数据也可能间接泄露。具体来说，当数据库中的数据点或记录之间存在关联时，查询结果仍然可能泄露响应之外记录的敏感信息。例如，记录之间的关联（例如跨组共享的相似属性或统计依赖关系）可能允许客户端推断出查询结果之外的其他敏感细节。这样，即使无法直接访问查询结果之外的记录，客户端也可能利用查询响应中的模式间接获取更广泛的数据集信息。这构成了一种数据隐私风险，需要谨慎处理。

为了解决这个问题，可以采用差分隐私技术[15, 26]来确保个人数据点不会被直接或间接泄露。本文暂不采用差分隐私，而是将其作为未来研究的方向。但我们注意到，将差分隐私集成到诸如 PoneglyphDB 之类的零知识证明（ZKP）系统中会增加电路设计的开销。

4 个定制大门

本节介绍用于在电路中表示 SQL 查询运算的定制化门电路。我们的目标是通过创建具有低阶多项式和较少电路约束的门电路来实现高效的设计。我们采用低阶多项式进行设计，是因为零知识证明 (ZKP) 依赖于密码学原语，而计算高阶多项式的计算成本很高。

4.1 射程检查

我们首先引入范围检查门，因为它涉及许多 SQL 操作，如“筛选”、“排序”、“分组”和“连接”。

考虑范围检查语句 $x \leq t$ ，其中 x 是私有值， t 是公共查询输入。一种简单的编码方式是使用多项式方程将 x 与每个可能的值进行比较：
$$\sum_{i=0}^t (x - i) = 0。$$

该多项式的次数随 t 线性增长，使得在 t 较大时，证明的生成和验证在计算上不可行。本文利用查找表 [19] 电路结构设计了范围校验门。使用查找表的直观思路在于预先计算并存储一系列可能输入的结果。我们的工作基于广泛使用的 Plookup 框架 [19]，该框架在零知识证明 (ZKP) 领域已得到充分应用。虽然存在其他替代方案，例如 Jolt [36]，但我们将这些方法的研究留待未来。我们并未重新发明密码学原语，而是专注于解决具体的实现难题，以提高 PLONK 框架内 SQL 操作的效率。虽然之前的研究 [30, 47] 使用布尔门或逻辑门框架实现了过滤、排序和连接等操作，但我们的方法则利用了基于算术的 PLONK 电路。这种根本性的差异带来了巨大的挑战，因为算术电路和布尔电路需要不同的优化策略。在这种情况下，调整 SQL 运算符需要重新思考它们的表达式和优化方式，以实现最高效率。

定义。范围检查门的输入是一个包含 n 个元素 $\{x, x, \dots, x\}$ 的列 C ，用于检查元素是否在 y 和 y 之间；输入是一个包含 m 个排序值 $\{y, y, \dots, y\}$ 的列（查找表） T ，表示有效范围，其中 y

设计 A：单范围检查。我们从一个简单的例子开始，即只证明单个值 x 位于特定范围 $[0, t]$ 内，从而证明 $0 \leq x \leq t$ 。在构建电路的第一步中，我们创建一个私有数组 P （存储在建议列中），其长度与集合 Q 相同。其中， P 的第一个元素是 x ，其余元素是从 Q 复制的任意值（这些值可以重复）。然后，证明器提供一个 P 的排列，记为 P ，其中 P 是私有的，存储在电路的建议列中。 P 中的值已排序，使得重复的值彼此相邻。

第二步，我们建立一个固定列来存储集合 Q ，并按升序排列这些值。这种组织方式确保证明者和验证者都能了解这些值及其在表中的对应索引。这些信息对于确定所需查找表的大小至关重要。

证明者提供一个 Q 的排列，记为 Q ，其中 Q 是私有的，存储在 PLONKish 电路的建议列中。排列 Q 的目的是隐藏 Q 中值的具体位置，因为验证者知道关于 Q 的所有信息。然后，电路可以比较 Q 和 P 中的值，以检查 x （在 P 中）是否等于 Q 中的某个值，而无需透露该值是 Q 中的哪个值。

现在，我们展示如何检查集合 P 中的值 x 是否等于集合 Q 中的某个值。集合 Q 中的值按特定顺序排列，使得 $P = Q$ 或 $P = P$ ，其中 P 和 Q 分别表示集合 P 和 Q 的第 i 个元素。具体来说，我们强制 P 和 Q 的前几个值相等，即当 $i = 0$ 时， $P = Q$ 。如果当 $i > 0$ 时， $P \neq Q$ ，则我们强制 $P = P$ ，这意味着在这种情况下， P 必须是 P 的副本。因此，这些约束保证了 P 中的每个值都等于 Q 中的某个值。形式上，我们使用以下规则强制规定 $P=Q$ 或 $P=P$ ：

$$0 = \begin{cases} (P-Q) \cdot (P-P) & \text{如果 } 1 \leq i \leq \text{len}(Q) - 1, P=Q \\ & \text{如果 } i = 0。 \end{cases} \tag{1}$$

有了上述多项式约束，验证者知道 P 的所有值都在 Q 中，而不知道 P 和 Q 中值的位置信息（因此它不知道 P 中每个位置的确切值，从而保护了 x 值的隐私）。

由于 P 和 Q 分别是 P 和 Q 的排列，上述多项式约束确保了 P （以及 P ）的所有值都在 Q 中。为了确保这一性质，我们设计多项式约束，以确保 P 是 P 的排列，且 Q 也是 Q 的排列：

$$\sum_{i=0}^{\text{len}(P)-1} (P+\alpha)(Q+\beta) = \sum_{i=0}^{\text{len}(Q)-1} (P+\alpha)(Q+\beta) \tag{2}$$

其中， P 、 Q 、 P 和 Q 分别表示 P 、 Q 、 P 和 Q 中的第 i 个元素， α 和 β 是随机选择的参数。随机值 α 和 β 隐藏了各列的内容，从而确保证验证过程中的保密性并防止出现零积。

	Advice column	Advice column	Advice column	Fixed column
	P	P'	Q'	Q
row 0:	x1=1	0	0	0
row 1:	x2=5	0	2	1
row 2:	0	x1=1	1	2
row 3:	0	x2=5	3	3

Figure 4: Range check with lookup tables illustration.

causing collisions due to poor randomness. To make the circuit formulation efficient, we express it as a recursive function to ensure that each equation maintains a low polynomial degree:

$$Z_{i+1} = Z_i \cdot \frac{(P_i + \alpha)(Q_i + \beta)}{(P'_i + \alpha)(Q'_i + \beta)} \quad (3)$$

$$Z_{\text{len}(Q)} = Z_0 = 1$$

Example 4.1. Figure 4 illustrates our proposed lookup table circuit design for proving a range check statement without revealing the specific values of x_1 and x_2 . The prover uses a fixed column Q accessible to verifiers, containing values in the range $[0,4)$. An advice column P includes the actual values of x_1 and x_2 with the remaining cells filled arbitrarily from Q . The prover then generates advice columns P' and Q' as permutations of P and Q , respectively, with P' sorted in ascending order and duplicates adjacent, ensuring Q' starts with the same value as P' . The prover checks that each P'_i is either equal to Q'_i or P'_{i-1} . This polynomial constraint ensures that values are within the range.

Design B: Batching range check. The lookup table technique facilitates verifying whether all elements of an array P belong to the set $Q = \{i \mid 0 \leq i \leq t, i \in \mathbb{Z}\}$, effectively checking if they fall within the range $[0, t]$. By organizing the elements of P into a single advice column and aligning the set Q within a fixed column, the lookup table approach previously discussed can be applied. For instance, the circuit depicted in Figure 4 can demonstrate whether the elements in the array $[0, 2, 1, 3]$ (column Q') are contained within the range $[0, 4)$ (column Q). The complexity of verifying the inclusion of the arrays $[0]$ and $[0, 1, 2, 3]$ within the range $[0, 4)$ remains consistent, as it necessitates the application of formulas 1 and 3 for each row.

Design C: Optimizing range checks with bitwise decomposition and lookup tables. To mitigate the scalability concerns associated with range checks, especially when the size of the set Q becomes significantly large (e.g., 2^{64}), we propose an optimization technique that leverages bitwise decomposition in conjunction with lookup tables. This method entails representing integers in a fixed bit-length format, such as 64 bits, and subsequently verifying the integrity of this bitwise representation in relation to the original integer value.

Given an integer N and its representation as a sequence of bits $b_0, b_1, \dots, b_{k-1}$, where b_0 is the least significant bit and b_{k-1} the most significant, the relationship between N and its bits is described by $N = \sum_{i=0}^{k-1} b_i \cdot 2^i$. We partition the bitwise representation of integer N into smaller segments of 8 bits, referred to as u8 cell. This approach is predicated on the standard binary representation of integers, wherein the integer is segmented into 8-bit blocks. For instance, a 64-bit integer is divided into 8 u8 cells, each encapsulating an 8-bit slice of the integer. Constraints are imposed to ensure this decomposition is accurately executed with the constraints $N = \sum_{i=0}^7 c_i \cdot 2^{8i}$ for a 64-bit integer N , where c_i represents the 8-bit segment of N at position i .

Each u8 cell c_i should have values within the range 0 to 255 (inclusive), corresponding to $2^8 - 1$. To validate each u8 cell, we utilize a fixed-sized lookup table of size 256, which contains integers from 0 to 255. This table allows efficient range checking for each segment of the integer. The advantage of using this lookup table is that it is fixed-sized (256 entries) and can be reused multiple times for each u8 cell check. This ensures that each u8 cell c_i can quickly verify if its value falls within the allowed range of 0 to $2^8 - 1$. By reusing the lookup table across all 8 u8 cells of N , we streamline the validation process and ensure consistent range checks.

Design D: Conditional statements proving. The previous range check method faces limitations when a value falls outside the lookup range, making proof construction difficult. To this end, we introduce an augmented method that seamlessly integrates with lookup tables to facilitate range checks while gracefully handling conditional scenarios. When dealing with data filtering, an upper-bound value, denoted as u , typically exists. Consequently, for $x \geq t$, this also implies $u > x \geq t$. Therefore, proving $u > x \geq t$ is equivalent to proving $0 \leq x - t \leq u$. To establish $x < t$, it suffices to demonstrate that $x - t < 0$. By adding u to both sides of the inequality, we obtain $0 \leq x - t + u < u$. Introducing a binary variable, denoted as *check*, to determine whether $x < t$, we ultimately prove the following statement:

$$0 \leq (x - t) + \text{check} \cdot u < u \quad (4)$$

To accomplish this, the prover configures several supporting columns in the PLONKish circuits. Initially, an advice column is created to output 1 if $x < t$ and 0 otherwise. Additionally, another advice column is established to store the values of $x - t$. Finally, the prover undertakes the task of proving that $\text{check} + (x - t)$ falls within the range of $[0, u)$ with the assistance of the lookup tables introduced in Section 4.1.

Note that the values in the “*check*” columns are prover-determined, with no explicit constraints imposed among x , t , and “*check*”. However, if the “*check*” values are inaccurately provided, the proof generation process encounters a failure. Moreover, the values in the “*x*”, “*t*”, and “*x-t*” columns adhere to the constraints $\text{cell}_i(x) - \text{cell}_i(t) - \text{cell}_i(x - t) = 0$ for the initial four rows, where $\text{cell}_i(x)$ represents the value in the “*x*” column at row i . These constraints guarantee that the discrepancies between corresponding elements in the “*x*” and “*t*” columns align with the values stored in the “*x-t*” column.

Correctness. We prove the correctness of the range check gate from two aspects. (1) Property 1: Element Inclusion. All the values in P' are in Q' . Assume that there exists one value x in P' that is not in Q' . We will show that this assumption leads to a contradiction. If x is the first value in P' , by Equation (1), $0 = P'_0 - Q'_0$, implying $P'_0 = Q'_0$. Thus, $x = P'_0$ contradicts $x \notin Q'$. If x is not the first, for $1 \leq i \leq \text{len}(Q') - 1$, $0 = (P'_i - Q'_i)(P'_i - P'_{i-1})$ enforces $P'_i = Q'_i$ or $P'_i = P'_{i-1}$. If $P'_i = Q'_i$, $x \in Q'$; if $P'_i = P'_{i-1}$, x duplicates P'_{i-1} , tracing back to $P'_0 = Q'_0$. Thus, x must be in Q' , contradicting $x \notin Q'$. Hence, all P' values are in Q' , proving Equation (1)’s constraints ensure $P' \subseteq Q'$. (2) Property 2: Permutation Integrity. Assume P' and Q' are not permutations of P and Q . This implies that some P_i or Q_i does not match any corresponding P'_i or Q'_i . Since the products involve symmetric polynomials, if $\{P'_i + \alpha\} \cup \{Q'_i + \beta\}$ are not permutations of $\{P_i + \alpha\} \cup \{Q_i + \beta\}$, the two sides of the equation cannot be equal due to the unique factorization of polynomials. Therefore, by the Fundamental Theorem of Symmetric Polynomials, P' and Q' must be permutations of P and Q .

	建议 柱子	建议 柱子	建议 柱子	固定的 柱子
	P	P'	Q'	Q
第 0 行:	x1=1	0	0	0
第 1 行:	x2=5	0	2	1
第 2 行:	0	x1=1	1	2
第 3 行:	0	x2=5	3	3

图 4: 使用查找表进行范围检查的示例。

由于随机性较差，导致碰撞。为了提高电路公式的效率，我们将其表示为递归函数，以确保每个方程都保持较低的多项式次数：

$$Z = Z \cdot \frac{(P+\alpha)(Q+\beta)}{(P+\alpha)(Q+\beta)} \quad (3)$$
$$Z = Z = 1$$

例 4.1。图 4 展示了我们提出的查找表电路设计，用于在不泄露 x 和 x 的具体值的情况下证明范围检查语句。证明器使用一个固定的列 Q ，验证器可以访问该列，其值范围为 $[0, 4)$ 。建议列 P 包含 x 和 x 的实际值，其余单元格则从 Q 中任意填充。然后，证明器生成建议列 P 和 Q ，它们分别是 P 和 Q 的排列，其中 P 按升序排列，重复的元素相邻排列，确保 Q 的初始值与 P 相同。证明器检查每个 P 是否等于 Q 或 P 。

此多项式约束确保数值在范围内。

设计方案 B：批处理范围检查。查找表技术

该方法便于验证数组 P 的所有元素是否都属于集合 $Q = \{i \mid 0 \leq i \leq t, i \in \mathbb{Z}\}$ ，从而有效地检查它们是否落在范围 $[0, t]$ 内。通过将 P 的元素组织到单个建议列中，并将集合 Q 对齐到固定列中，即可应用前面讨论过的查找表方法。例如，图 4 所示的电路可以演示数组 $[0, 2, 1, 3]$ （列 Q ）中的元素是否包含在范围 $[0, 4)$ （列 Q ）内。验证数组 $[0]$ 和 $[0, 1, 2, 3]$ 是否包含在范围 $[0, 4)$ 内的复杂度保持不变，因为它需要对每一行应用公式 1 和 3。

设计 C：利用按位分解优化范围检查

以及查找表。为了缓解范围检查带来的可扩展性问题，尤其是在集合 Q 的大小变得非常大（例如，2）时，我们提出了一种优化技术，该技术结合了按位分解和查找表。该方法首先将整数表示为固定位长度的格式（例如 64 位），然后验证该按位表示与原始整数值的一致性。给定一个整数 N 及其比特序列 $b, b, \dots, b$ ，其中 b 是最低有效位， b 是最高有效位， N 与其比特之间的关系由下式描述：

$N = \sum_{i=0}^{\ell-1} b_i \cdot 2^i$ 。我们将整数 N 的按位表示分割成更小的 8 位段，称为 u8 单元。这种方法基于整数的标准二进制表示，其中整数被分割成 8 位块。例如，一个 64 位整数被分割成 8 个 u8 单元，每个单元封装整数的 8 位切片。施加约束以确保这种分解能够准确执行，约束条件为 $N =$

对于 64 位整数 N ，
 $c \cdot 2$ 表示 N 中位置
 i 处的 8 位段。

每个 u8 单元格 c 的值应在 0 到 255（含 0 和 255）的范围内，对应于 2^{-1} 。为了验证每个 u8 单元格，我们使用了一个大小为 256 的固定查找表，其中包含 0 到 255 的整数。该查找表可以高效地检查整数的每个区间。使用此查找表的优势在于其大小固定（256 个条目），并且可以重复用于每个 u8 单元格的检查。这确保了每个 u8 单元格 c 可以快速验证其值是否在 0 到 2^{-1} 的允许范围内。通过在 N 的所有 8 个 u8 单元格中重复使用该查找表，我们简化了验证过程并确保了范围检查的一致性。

设计 D：条件语句证明。先前的范围

当值超出查找范围时，检查方法会面临局限性，导致证明困难。为此，我们引入了一种增强方法，该方法与查找表无缝集成，便于进行范围检查，并能优雅地处理各种条件场景。在处理数据过滤时，通常存在一个上限值，记为 u 。因此，对于 $x \geq t$ ，这也意味着 $u > x \geq t$ 。所以，证明 $u > x \geq t$ 等价于证明 $0 \leq x - t \leq u$ 。为了证明 $x < t$ ，只需证明 $x - t < 0$ 。通过在不等式两边加上 u ，我们得到 $0 \leq x - t + u < u$ 。引入一个二元变量，记为 $check$ ，用于判断 x 是否 $< t$ ，我们最终证明了以下命题：

$$0 \leq (x - t) + check \cdot u < u \quad (4)$$

为了实现这一点，证明器在 PLONKish 电路中配置了几个辅助列。首先，创建一个建议列，当 $x < t$ 时输出 1，否则输出 0。此外，还创建了另一个建议列来存储 $x - t$ 的值。最后，证明器借助 4.1 节中介绍的查找表，证明 $check + (x - t)$ 的值在 $[0, u)$ 范围内。

请注意，“check”列中的值由证明者确定， x 、 t 和“check”之间没有显式约束。但是，如果“check”值提供不准确，则证明生成过程会失败。此外，对于前四行，“x”、“t”和“xt”列中的值满足约束条件： $cell(x) - cell(t) - cell(x - t) = 0$ ，其中 $cell(x)$ 表示第 i 行“x”列中的值。这些约束保证了“x”列和“t”列中对应元素之间的差异与“xt”列中存储的值一致。

正确性。我们从两个方面证明范围校验门的正确性。(1) 性质 1：元素包含性。P 中的所有值都在 Q 中。假设 P 中存在一个值 x 不在 Q 中。我们将证明此假设会导致矛盾。如果 x 是 P 中的第一个值，根据等式 (1)， $0 = P - Q$ ，这意味着 $P = Q$ 。因此， $x = P$ 与 $x \notin Q$ 矛盾。如果 x 不是第一个值，对于 $1 \leq i \leq \text{len}(Q) - 1$ ， $0 =$

$(P - Q)(P - P)$ 强制 $P = Q$ 或 $P = P$ 。如果 $P = Q$ ，则 $x \in Q$ ；如果 $P = P$ ，则 x 与 P 重复，由此可追溯到 $P = Q$ 。因此， x 必须属于 Q ，这与 $x \notin Q$ 矛盾。因此，所有 P 值都属于 Q ，证明了等式 (1) 的约束条件确保 $P \subseteq Q$ 。(2) 性质 2：置换完整性。假设 P 和 Q 不是 P 和 Q 的置换。这意味着某些 P 或 Q 不与任何对应的 P 或 Q 匹配。由于乘积涉及对称多项式，如果 $\{P + \alpha\} \cup \{Q + \beta\}$ 不是 $\{P + \alpha\} \cup \{Q + \beta\}$ 的排列，则由于多项式分解的唯一性，等式两边不可能相等。因此，根据对称多项式基本定理， P 和 Q 必定是 P 和 Q 的排列。

With the two properties proven above, we can guarantee the correctness of proving $x < t$ if x is not larger than t . The inequality in Equation 4 holds if $check = 1$ and $x < t$, or if $check = 0$ and $x \leq t$. Therefore, setting the binary variable $check$ correctly is sufficient to determine whether $x < t$ or not. By transforming x to $(x-t)+check \cdot u$, we can guarantee that the transformed x is in the range of 0 to u .

Complexity of a Range Check Gate. The ZK proof generation cost depends on the number of constraints. We analyze the complexity of a gate by specifying the required number of each type of constraint. To validate that all elements of an array P reside within a set Q , the number of constraints, as defined in Equations 1 and 3, corresponds to the greater of $|P|$ or $|Q|$, denoted as $\max(|P|, |Q|)$. When employing the bitwise decomposition method for an input set P where $|P| > 256$, the lookup table is padded to match the size of the input by including duplicates of values ranging from 0 to 255. For 64-bit integers decomposed into 8 u8 cells, each cell undergoes a range check utilizing the lookup table. The number of constraints, as described by Equations 1 and 3, is equivalent to $8|P|$. Additionally, the number of constraints required to ensure the correct decomposition of integers into u8 cells, as well as verifying that all segments of the integer fall within the specified ranges, is $|P|$. Finally, to transform a value x with $x' = (x-t)+check \cdot u$ (see Equation 4), an additional $|P|$ constraints are needed.

4.2 Sort

We detail our approach to proving the correctness of sort operations.

Definition. The input of a sort gate consists of a table D with m columns $C_1, C_2, \dots, C_m$, where C_i represents different attributes or values to be sorted. Each column C_i contains n elements $\{x_{i1}, x_{i2}, \dots, x_{in}\}$. The output is a table SD with m columns $C'_1, C'_2, \dots, C'_m$, where each column C'_i contains the sorted elements $\{y_{i1}, y_{i2}, \dots, y_{in}\}$. Here, y_{ij} represents the j -th element in column C'_i of D , arranged in the desired order (e.g. increasing or decreasing).

Design. The first step in proving sort operations entails generating a witness, which includes the result of applying a specific sorting algorithm to the input data. The prover has the flexibility to choose any sorting algorithm, as long as the resulting order is correct. Let D and R represent the input data and the sorted result, respectively. Two essential properties of R must be guaranteed for a successful proof. First, the data in R should match that in D except for the order, leading to a permutation check between R and D . The following constraints, akin to Equations (3), ensure this:

$$\begin{aligned} Z_{i+1} &= Z_i \cdot \frac{R_i + \alpha}{D_i + \alpha} \\ Z_{\text{len}(D)} &= Z_0 = 1 \end{aligned} \quad (5)$$

Here, D_i and R_i represent the i -th element in D and R , and α is a randomly chosen parameter similar to ones in Equation 2.

In sorting mechanisms where multiple attributes are considered, a unified approach is adopted to encapsulate these attributes into a singular composite entity. This is achieved by allocating a consistent bit-length representation for each attribute, specifically employing a 64-bit format for this purpose. Such a fixed bit-length representation is critical in preserving the intrinsic value hierarchy and relative ordering of each attribute during the process of concatenation. In addition, the data in R must align with the sort definition. To verify this, we check that $R_i \leq R_{i+1}$ (assuming an ascending order) for i in the range $[0, \text{len}(R) - 1]$. This is achieved by proving the transformed statement introduced in Equation 4 with the assistance of lookup tables.

Correctness. We prove the correctness of the two properties introduced in the sort gate. (1) Property 1: Permutation Integrity. R is a permutation of D . The proof follows a similar approach used to prove that P' is a permutation of P . For details, refer to the proof in Section 4.1. (2) Property 2: Sortedness. R is sorted in ascending order. Assume, for contradiction, that R is not sorted in ascending order. This implies there exists at least one pair of indices $i < j$ such that $R_i > R_j$. Since we enforce the constraints $R_i \leq R_{i+1}$ for each pair (R_i, R_{i+1}) where $i \in [0, \text{len}(R) - 1]$, and the correctness of the range check gate for $R_i \leq R_{i+1}$ is proven in Section 4.1, we guarantee that $R_i \leq R_{i+1}$ holds for all valid indices i . This contradicts the assumption that there exists at least one pair of indices $i < j$ such that $R_i > R_j$. Therefore, R must indeed be sorted in ascending order to satisfy the defined properties and constraints of the sort operation.

Complexity of a sort gate. Permutation checks between D and R require $|D|$ constraints, as per Equation 5. Additionally, checking $R_i \leq R_{i+1}$ for $i \in [0, \text{len}(R) - 1]$ requires constraints proportional to $|D|$, similar to the range check gate in Section 4.1.

4.3 Group-by

In this section, we outline our methodology for verifying the correctness of group-by operations.

Definition. The input of a group-by gate consists of a table D with n columns $C_1, C_2, \dots, C_n$, where each column represents different attributes of D . The output is a table SD with $n+2$ columns that rearranges the records of D such that records with identical values on the grouping attributes G are placed into the same group-by bin. Alongside the original n columns from D , SD includes two additional columns: (1) `start_index`: Indicates the starting index of each group-by bin in D . (2) `end_index`: Indicates the ending index of each group-by bin in D . The `start_index` and `end_index` are used for the later aggregation functions such as SUM and others.

Design. Given an input table D and the group-by attributes G , the prover first generates the sorted table SD based on the group-by attributes G . This sorting ensures that records with identical values in G are adjacent in SD . To verify that SD is a sorted version of D , we employ the approach introduced in Section 4.2.

To identify the starting and ending indices of each group-by bin, we check each record in SD on the group-by attributes G . We use a binary value b to indicate whether a record in SD is a starting or ending record, where 1 signifies that the record is either a starting or ending record. The constraint to check whether two values v_1 and v_2 are equal or not with a binary value b is as follows:

$$b = 1 - (v_1 - v_2) \cdot p \quad (6)$$

where p is the value provided by the prover. Specifically, $p = 0$ if $v_1 = v_2$ and $p = \frac{1}{v_1 - v_2}$ otherwise. To ensure that the prover provides the correct value of p , we add the following constraint for each pair of v_1 and v_2 :

$$b \cdot (v_1 - v_2) = 0 \quad (7)$$

A record is marked as the start of its bin if no preceding adjacent records share identical values in group-by attributes G . Conversely, it is marked as the end of its bin if no subsequent adjacent records share identical values in G .

Correctness. We utilize the sort gate introduced in Section 4.2 to ensure that SD is sorted. The correctness of the sort gate is demonstrated in Section 4.2. Next, we deduce the correctness of the starting and ending indices of each group-by bin. According to Equation 6,

基于上述两个性质的证明, 我们可以保证当 x 不大于 t 时, 证明 $x < t$ 的正确性。当 $check = 1$ 且 $x < t$, 或者当 $check = 0$ 且 $x \leq t$ 时, 等式 4 中的不等式成立。因此, 正确设置二元变量 $check$ 足以确定 $x < t$ 是否成立。通过将 x 变换为 $(x - t) + check \cdot u$, 我们可以保证变换后的 x 的取值范围为 0 到 u 。

范围校验门的复杂性。零知识证明生成
成本取决于约束的数量。我们通过指定每种约束所需的数量来分析门的复杂度。为了验证数组 P 的所有元素都位于集合 Q 内, 约束的数量 (如公式 1 和 3 所定义) 对应于 $|P|$ 或 $|Q|$ 中的较大值, 记为 $\max(|P|, |Q|)$ 。当对输入集合 P 使用按位分解方法, 且 $|P| > 256$ 时, 查找表会通过包含 0 到 255 之间值的重复项来填充, 以匹配输入的大小。对于分解为 8 个 u8 单元格的 64 位整数, 每个单元格都会使用查找表进行范围检查。如公式 1 和 3 所述, 约束的数量等于 $8|P|$ 。此外, 为确保将整数正确分解为 u8 单元格, 并验证整数的所有段均落在指定范围内, 所需的约束数量为 $|P|$ 。最后, 要使用 $x = (x - t) + check \cdot u$ (参见公式 4) 转换值 x , 还需要额外的 $|P|$ 个约束。

4.2 排序

我们详细介绍了证明排序操作正确性的方法。

定义。 排序门的输入是一个包含 m 列的表 D , 列分别为 $C_1, C_2, \dots, C_n$, 其中 C 代表待排序的不同属性或值。每列 C 包含 n 个元素 $\{x_1, x_2, \dots, x_n\}$ 。输出是一个包含 m 列的表 SD , 列分别为 $C_1, C_2, \dots, C_n$, 其中每列 C 包含已排序的元素 $\{y_1, y_2, \dots, y_n\}$ 。这里, y_n 表示 D 中 C 列的第 j 个元素, 并按所需顺序 (例如, 升序或降序) 排列。

设计。 证明排序操作的第一步是生成一个见证, 其中包含对输入数据应用特定排序算法的结果。证明者可以灵活选择任何排序算法, 只要结果顺序正确即可。令 D 和 R 分别表示输入数据和排序结果。为了成功证明, 必须保证 R 的两个基本属性。首先, R 中的数据应与 D 中的数据匹配, 但顺序不同, 这导致 R 和 D 之间存在置换检查。以下约束 (类似于公式 (3)) 确保了这一点:

$$\begin{aligned} Z &= Z \cdot \frac{R + \alpha}{D + \alpha} \\ Z &= Z \cdot 1 \end{aligned} \quad (5)$$

这里, D 和 R 分别表示 D 和 R 中的第 i 个元素, α 是一个随机选择的参数, 类似于公式 2 中的参数。

在考虑多个属性的排序机制中, 采用统一的方法将这些属性封装成一个单一的复合实体。具体而言, 为每个属性分配一致的位长度表示, 并采用 64 位格式。这种固定的位长度表示对于在连接过程中保持每个属性的固有值层次结构和相对顺序至关重要。此外, R 中的数据必须与排序定义一致。为了验证这一点, 我们检查对于 $[0, \text{len}(R) - 1]$ 范围内的 i , $R \leq R$ (假设升序)。这是通过借助查找表证明公式 4 中引入的转换语句来实现的。

正确性。 我们证明了排序门中引入的两个性质的正确性。(1) 性质 1: 置换完整性。 R 是 D 的一个置换。证明方法与证明 P 是 P 的一个置换的方法类似。详情请参见 4.1 节的证明。(2) 性质 2: 有序性。 R 按升序排列。假设 R 不是按升序排列, 以证明矛盾。这意味着至少存在一对索引 $i < j$, 使得 $R_i > R_j$ 。由于我们对每对索引 $i \in [0, \text{len}(R) - 1]$ 都强制执行约束 $R_i \leq R_j$, 并且第 4.1 节证明了范围检查门对 $R_i \leq R_j$ 的正确性, 因此我们保证对于所有有效的索引 i, j , $R_i \leq R_j$ 都成立。这与至少存在一对索引 $i < j$ 使得 $R_i > R_j$ 的假设相矛盾。因此, R 必须按升序排列才能满足排序操作的定义性质和约束。

排序门的复杂度。 根据公式 5, D 和 R 之间的置换检查需要 $|D|$ 个约束。此外, 检查 $R_i \leq R_j$ (其中 $i \in [0, \text{len}(R) - 1]$) 需要与 $|D|$ 成比例的约束, 类似于 4.1 节中的范围检查门。

4.3 分组

在本节中, 我们将概述我们验证分组操作正确性的方法。

定义。 分组门的输入是一个包含 n 列的表 D , 列分别为 $C_1, C_2, \dots, C_n$, 其中每一列代表 D 的不同属性。输出是一个包含 $n + 2$ 列的表 SD , 它重新排列 D 中的记录, 使得分组属性 G 值相同的记录被放入同一个分组区间。除了 D 的原始 n 列之外, SD 还包含两个附加列: (1) start_index : 指示 D 中每个分组区间的起始索引。(2) end_index : 指示 D 中每个分组区间的结束索引。 start_index 和 end_index 用于后续的聚合函数, 例如 SUM 等。

设计。 给定输入表 D 和分组属性 G , 证明器首先基于分组属性 G 生成排序表 SD 。这种排序确保 G 中具有相同值的记录在 SD 中相邻。为了验证 SD 是否为 D 的排序版本, 我们采用第 4.2 节中介绍的方法。

为了确定每个分组区间的起始索引和结束索引, 我们检查 SD 中的每个记录在分组属性 G 上的值。我们使用二进制值 b 来指示 SD 中的记录是起始记录还是结束记录, 其中 1 表示该记录是起始记录或结束记录。使用二进制值 b 检查两个值 v 和 v' 是否相等的约束如下:

$$b = 1 - (v - v') \cdot p \quad (6)$$

其中 p 是证明者提供的值。具体来说, 如果 $v = v'$, 则 $p = 0$; 如果 $v \neq v'$, 则 $p = 1$ 。为了确保证明器提供正确的 p 值, 我们对每一对 v 和 v' 添加以下约束:

$$b \cdot (v - v') = 0 \quad (7)$$

如果前面的相邻记录在分组属性 G 中没有相同的值, 则该记录被标记为其所在区间的开始。相反, 如果后面的相邻记录在 G 中没有相同的值, 则该记录被标记为其所在区间的结束。

正确性。 我们利用第 4.2 节中介绍的排序门来确保 SD 已排序。排序门的正确性已在第 4.2 节中证明。接下来, 我们推导出每个分组箱的起始索引和结束索引的正确性。根据公式 6,

Input table: D		Sorted D		Start	End	Output	
D1	D2	SD1	SD2	S	E	M	O
1	2	1	2	1	0	2	12
3	6	1	10	0	1	2+10	8
2	8	2	8	1	1	8	6
1	10	3	6	1	1	6	

Figure 5: Illustration with the SQL query “SELECT SUM(D2) FROM T GROUP BY D1.”

if $v_1 = v_2$, then $b = 1$, and Equation 7 holds trivially as $b \cdot (v_1 - v_2) = 0$. If $v_1 \neq v_2$: (1) If $p = \frac{1}{v_1 - v_2}$, then $b = 0$, and Equation 7 holds because $b \cdot (v_1 - v_2) = 0$. (2) If $p = 0$, then $b = 1$. In this case, Equation 7 does not hold as $b \cdot (v_1 - v_2) = (v_1 - v_2) \neq 0$. Consequently, a valid proof cannot be generated because the system detects the inconsistency.

Complexity of a group-by gate. The group-by operation is facilitated through the sorting of associated attributes, followed by a verification of the accuracy of sorting. Consequently, the computational complexity and the number of constraints required for ensuring the correctness of group-by operations are identical to those identified in the analysis of sorting constraints. Additionally, the number of constraints (Equations 6 and 7) needed to identify the starting and ending indices of each group-by bin is $2|D|$ where D is the input set.

Example 4.2. Figure 5 illustrates the SQL query "SELECT SUM(D2) FROM T GROUP BY D1." The input table has columns D1 and D2. The prover adds sorted columns SD1 and SD2, as well as columns S and E for group-by bin indices. An advice column M is created where M_i is defined as $M_{i-1} + SD2_i$ if $SD1_i = SD1_{i-1}$, otherwise $M_i = SD2_i$, with $M_0 = SD2_0$. The final output column O captures the result by copying only the last record of each group-by bin, as indicated by the E column.

4.4 Join

Now, we present our methodology for verifying the correctness of join operations.

Definition. The input of a join gate consists of two tables $T1$ and $T2$, each represented by m columns $C_1^{(1)}, C_2^{(1)}, \dots, C_m^{(1)}$ for $T1$ and n columns $C_1^{(2)}, C_2^{(2)}, \dots, C_n^{(2)}$ for $T2$. These columns correspond to different attributes of $T1$ and $T2$, respectively. The joining operation is performed based on equality conditions between specified joining attributes $J1$ from $T1$ and $J2$ from $T2$, which may involve any of the columns $C_i^{(1)}$ from $T1$ and $C_j^{(2)}$ from $T2$. The output is a table JD that combines records from $T1$ and $T2$ where the joining condition $T1.J1 = T2.J2$ is satisfied. The table JD has $m+n$ columns: (1) The first m columns $C_1^{(1)}, C_2^{(1)}, \dots, C_m^{(1)}$ are the attributes from $T1$. (2) The next n columns $C_1^{(2)}, C_2^{(2)}, \dots, C_n^{(2)}$ are the attributes from $T2$.

Design. Consider two private tables $T1$ and $T2$, and let p be the join predicate. To establish the correctness of the join operations, the prover calculates several types of witnesses locally. Initially, the prover creates two new tables $T'1$ and $T'2$ to reorder the records within them. Specifically, each table $T'1$ and $T'2$ is split into two parts: $T'1_p$ and $T'2_p$ contain records contributing to the join predicate, while $T'1_{non-p}$ and $T'2_{non-p}$ contain records that do not contribute to the join predicate. The correctness of this reordering is verified using polynomial constraints, as detailed in Equation 5.

T1		T2		T1'		T2'		Deduplicated tables				Sorted table		Join result			
D1	D2	D1'	D2'	GD1	GD2	GD1'	GD2'	S1	S1'	S2	S2'	S	J1	J2	J1'	J2'	
1	1	3	11	1	1	1	12	6	6	5	4	4	1	1	1	12	
3	2	1	12	1	4	3	11	6		4	5	5	1	4	1	12	
6	3	5	13	3	2	5	13			7	7	6	3	2	3	11	
1	4	4	14	6	3	4	14					7					
6	5	7	15	6	5	7	15										

Figure 6: Illustration with the SQL query “SELECT T1.D1, T2.D2' FROM T1, T2 WHERE T1.D1 = T2.D1' ”.

Next, the prover ensures that the records in $T'1$ and $T'2$ indeed contribute to the join predicate. In the context of primary key-foreign key joins (non-primary key-foreign key joins are discussed later), assuming that table $T1$ contains the foreign key and table $T2$ contains the primary key, instead of checking the existence of records in one table in another, the prover proves that $T'2_{non-p}$ is disjoint from $T'1_{non-p}$ and $T'1_p$. Traditional straight-forward methods involve checking each value in $T'1_{non-p}$ against $T'2_{non-p}$, resulting in a large number of polynomial constraints. To address this, a sorted table S is created to store the unique values in $T'1_{non-p}$ and $T'2_{non-p}$, with a proof of $S_i < S_{i+1}$ for $i \in [0, \text{len}(S) - 1]$.

However, duplicates¹ in S pose challenges in determining their origin (from the same or different tables). To resolve this, the prover implements a deduplication strategy through the creation of distinct versions, namely $T1^{de}$ and $T2^{de}$. This process ensures that each value in $T'1_{non-p}$ is accounted for in $T1^{de}$ and every value in $T'2_{non-p}$ is accounted for in $T2^{de}$. To establish this deduplication property, we leverage the mechanism of lookup tables we proposed in Section 4.1. The verification of the range check operation, which confirms the existence of each value in a column within a lookup table (i.e. another column), is adapted to ensure the deduplication of $T1^{de}$ and $T2^{de}$. It is noteworthy that the distinction between $T1^{de}$, $T2^{de}$, and a lookup table for range check lies in their roles within the PLONKish circuit configuration. Specifically, $T1^{de}$ and $T2^{de}$ serve as advice columns, signifying the privacy of the data they contain, while a lookup table for range check is stored in an instance column, designating the public nature of the data within it. This distinction is integral to the overall architecture of the PLONKish circuit configuration. The prover establishes that S is a permutation of $T1^{de} \cup T2^{de}$, ensuring that S matches the records in $T1^{de}$ and $T2^{de}$ except for the order. And $S_i < S_{i+1}$ holds for $\forall i \in [0, \text{len}(S) - 1]$.

Next, we introduce the method for generating join results from contributing records. Consider the scenario where the join predicate between two tables, $T1$ and $T2$, is defined as $T1.attr1 = T2.attr2$. In the context of primary key-foreign key joins, the uniqueness of primary keys implies the absence of duplicates, ensuring that each foreign key in $T1$ corresponds to at most one matching row in $T2$. Let us denote $T'1_p$ and $T'2_p$ as the subsets of $T1$ and $T2$, respectively, that are relevant to the join predicate. Within $T1$, $attr1$ serves as the foreign key, whereas $attr2$ is the primary key within $T2$. For each record in $T'1_p$, a search is conducted within $T'2_p$ to identify any record that satisfies the

¹ Since SQL operates on multisets, duplicates are not only a concern in join algorithms but also throughout the entire query process, as the result must accurately reflect the correct number of duplicates. In Plookup [19], this can be efficiently managed by encoding the frequency of elements in the multiset into polynomial commitments. Plookup then ensures that the correct number of occurrences for each value is preserved during query execution, maintaining the semantic integrity of SQL operations.

输入表: D		已排序 D		开始		结尾	输出
D1	D2	SD1	SD2	S	E	M	O
1	2	1	2	1	0	2	12
3	6	1	10	0	1	2+10	8
2	8	2	8	1	1	8	6
1	10	3	6	1	1	6	

图 5: SQL 查询 “SELECT SUM(D2) FROM T GROUP BY D1” 的示例。

如果 $v=v$, 则 $b=1$, 且等式 7 显然成立, 因为 $b \cdot (v-v)=0$ 。如果 $v \neq v$, 则 $b=0$, 且等式 7 成立, 因为 $b \cdot (v-v)=0$ 。(2) 如果 $p=0$, 则 $b=1$ 。在这种情况下, 等式 7 不成立, 因为 $b \cdot (v-v)=(v-v) \neq 0$ 。因此, 由于系统检测到不一致, 无法生成有效的证明。

分组门的复杂性。分组操作通过对关联属性进行排序, 然后验证排序的准确性来实现。因此, 确保分组操作正确性所需的计算复杂度和约束数量与排序约束分析中确定的相同。此外, 确定每个分组箱的起始索引和结束索引所需的约束数量 (公式 6 和 7) 为 $2|D|$, 其中 D 为输入集。

例 4.2。图 5 展示了 SQL 查询 “SELECT SUM(D2) FROM T GROUP BY D1”。输入表包含列 D1 和 D2。验证器添加了排序后的列 SD1 和 SD2, 以及用于分组索引的列 S 和 E。创建了一个建议列 M, 其中 M 定义为: 如果 $SD1=SD1$, 则 $M+SD2$; 否则 $M=SD2$ 。最终输出列 O 通过仅复制每个分组索引的最后一记录来捕获结果, 如 E 列所示。

4.4 加入

现在, 我们介绍验证连接操作正确性的方法。

定义。连接门的输入由两个表 T_1 和 T_2 组成, 每个表由 m 列 C 表示。

对于 T_1 , $C_1, C_2, \dots, C_m$ 对于 T_1 和 T_2 , 第 1、2、3、4 列分别对应于 T_1 和 T_2 的不同属性。连接操作基于 T_1 中的指定连接属性 J_1 和 T_2 中的 J_2 之间的相等条件执行, 这些条件可能涉及第 1、2、3 列中的任何一列。

分别从 T_1 和 C 中获取记录, 从 T_2 中获取记录。输出结果是一个表 J , 它合并了来自 T_1 和 T_2 的记录, 其中连接条件 $T_1.J_1 = T_2.J_2$ 成立。表 J 有 $m+n$ 列: (1) 前 m 列 $C_1, C_2, \dots, C_m$ 是来自 T_1 的属性。(2) 该

接下来 n 列 $C_{m+1}, C_{m+2}, \dots, C_{m+n}$ 是来自 T_2 的属性。设计。考虑两个私有表 T_1 和 T_2 , 令 p 为连接谓词。为了验证连接操作的正确性, 证明器在本地计算几种类型的见证。首先, 证明器创建两个新表 T_1 和 T_2 , 并重新排列其中的记录。具体来说, 每个表 T_1 和 T_2 都被拆分为两部分: 一部分包含构成连接谓词的记录, 另一部分包含不构成连接谓词的记录。这种重新排序的正确性使用多项式约束进行验证, 详见公式 5。

T1		T2		T1'		T2'		Duplicated tables				Sorted table		Join result			
D1	D2	D1'	D2'	GD1	GD2	GD1'	GD2'	S1	S1'	S2	S2'	S	J1	J2	J1'	J2'	
1	1	3	11	1	1	1	12	6	6	5	4	4	1	1	1	12	
3	2	1	12	1	4	3	11	6		4	5	5	2	4	3	12	
6	3	5	13	3	2	5	13			7	7	6	3	2	5	11	
1	4	4	14	6	3	4	14					7					
6	5	7	15	6	5	7	15										

图 6: SQL 查询 “SELECT T1.D1, T2.D2' FROM T1, T2 WHERE T1.D1 = T2.D1'” 的示意图。

接下来, 证明器确保表 T_1 和 T_2 中的记录确实对连接谓词有贡献。在主键-外键连接的上下文中 (非主键-外键连接将在后面讨论), 假设表 T_1 包含外键, 表 T_2 包含主键, 证明器无需检查一个表中的记录是否存在于另一个表中, 而是证明 T_2 与 T_1 和 T_2 不相交。传统的直接方法需要将 T_1 中的每个值与 T_2 进行比较, 从而导致大量的多项式约束。为了解决这个问题, 我们创建了一个排序表 S 来存储 T_1 和 T_2 中的唯一值, 并证明对于 $i \in [0, \text{len}(S) - 1]$, $S_i < S_{i+1}$ 。

然而, S 中的重复项给确定其来源 (来自同一张表还是不同的表) 带来了挑战。为了解决这个问题, 证明器通过创建不同的版本 T_1 和 T_2 来实现去重策略。此过程确保 T_{1n-p} 中的每个值都在 T_1 中得到对应, 并且 T_{2n-p} 中的每个值都在 T_2 中得到对应。为了建立这种去重特性, 我们利用了我们在 4.1 节中提出的查找表机制。范围检查操作的验证旨在确认查找表 (即另一列) 中每一列的值是否存在, 并经过调整以确保 T_1 和 T_2 的去重。值得注意的是, T_1 、 T_2 和范围检查查找表之间的区别在于它们在 PLONK 式电路配置中的角色。具体来说, T_1 和 T_2 作为建议列, 表明它们包含的数据是私密的; 而范围检查查找表则存储在实例列中, 表明其包含的数据是公开的。这种区别是 PLONK 式电路配置整体架构不可或缺的一部分。证明器证明 S 是 $T_1 \cup T_2$ 的一个排列, 从而确保 S 与 T_1 和 T_2 中的记录匹配, 只是顺序不同。且对于 $\forall i \in [0, \text{len}(S) - 1]$, $S_i < S_{i+1}$ 成立。

接下来, 我们将介绍从贡献记录生成连接结果的方法。考虑两个表 T_1 和 T_2 之间的连接谓词定义为 $T_1.\text{attr1} = T_2.\text{attr2}$ 的情况。在主键-外键连接的上下文中, 主键的唯一性意味着不存在重复项, 从而确保 T_1 中的每个外键在 T_2 中至多对应一个匹配行。我们将 T_1 和 T_2 分别表示为 T_1 和 T_2 中与连接谓词相关的子集。在 T_1 中, attr1 作为外键, 而在 T_2 中, attr2 作为主键。对于 T_1 中的每条记录, 在 T_2 中执行搜索, 以识别满足连接谓词的任何记录。

由于 SQL 操作的是多重集, 重复项不仅在连接算法中需要关注, 而且贯穿整个查询过程, 因为结果必须准确反映重复项的正确数量。在 Plookup 函数 [19] 中, 可以通过将多重集中元素的频率编码为多项式承诺来有效地处理这个问题。Plookup 函数随后确保在查询执行期间每个值的出现次数都正确, 从而维护 SQL 操作的语义完整性。

join predicate. Upon finding a match, the record from $T2'_p$ is concatenated with the corresponding record from $T1'_p$, forming a combined record. To verify the join results, two key properties must be established: (1) **Equality Verification**: For each concatenated record r , we set the polynomial $r.attr1 - r.attr2 = 0$, where $attr1$ and $attr2$ are the attributes from $T1$ and $T2$, respectively, used for the join. (2) **Source Verification**: To ensure all records joined with $T1'_p$ originate exclusively from $T2'_p$, we use lookup tables as described in Section 4.1.

To handle joins without relying on primary key-foreign key relationships, we compute the join result by pairwise comparing records from $T1'_p$ and $T2'_p$ using a similar procedure. However, the method introduced above may not include all records contributing to the join predicate in $T1'$ and $T2'$. To ensure completeness, we additionally prove that $T1'_{non-p}$ is disjoint from $T2'$ using the same deduplication and sorting mechanism. This ensures that all relevant records are considered in the join operation.

Scalability. When managing a large number of join operations, the database's query engine typically executes these joins sequentially, joining two tables at a time. This results in various execution plans having differing numbers of total and intermediate join results. Optimizations aimed at reducing the number of these total and intermediate join results should be identified and applied prior to the circuit design phase.

Correctness. To ensure the correctness of the join gate, we verify the following properties: (1) Property 1: Permutation Integrity. The sets $T1'_p \cup T1'_{non-p}$ and $T2'_p \cup T2'_{non-p}$ are permutations of the original tables $T1$ and $T2$, respectively. The proof follows a similar approach used to prove that P' is a permutation of P . For details, refer to the proof in Section 4.1. (2) Property 2: Completeness. $T1'$ and $T2'$ include all records that contribute to the join predicate. Assume $T1'$ misses a record x that contributes to the join predicate, i.e., x is in $T1'_{non-p}$. We prove that $T1'_{non-p}$ is disjoint from $T2'$ and $T2'_{non-p}$ using the Element Inclusion and Sortedness properties as detailed in Section 4.2. This implies x does not overlap with $T2'$, indicating that x does not contribute to the join predicate. This contradicts the assumption, proving that $T1'$ includes all relevant records. Similarly, $T2'$ includes all records that contribute to the join predicate. (3) Property 3: Exclusivity. $T1'$ and $T2'$ do not include records that do not contribute to the join predicate. Assume $T1'$ includes a record y that does not satisfy the join condition. During the join process, the concatenated record r involving y would fail the constraint $r.attr1 - r.attr2 = 0$, contradicting the assumption. Therefore, $T1'$ and $T2'$ only include records that contribute to the join predicate. The correctness of the above three properties ensures the correctness of the join gate.

Example 4.3. Figure 6 illustrates the SQL query "SELECT T1.D1, T2.D2 FROM T1, T2 WHERE T1.D1 = T2.D1'." Upon receiving the input tables $T1$ and $T2$, the prover creates new tables $T1'$ and $T2'$. The upper parts contain records contributing to the join $T1.D1 = T2.D1'$. The green area in columns $GD1$ and $GD1'$ shows corresponding values. The prover verifies that the non-contributing records (gray areas) in $GD1$ and $GD1'$ do not intersect by constructing columns $S1$ and $S2$, removing duplicates, and ensuring that each value in $S1$ and $S2$ exists in their respective sorted columns $S1'$ and $S2'$. The process of proving that the non-contributing records (gray areas) in $GD1'$ do not intersect with the contributing records (green areas) in $GD1$ is performed in a similar manner. This step is omitted in the example and assumes that

table $T1$ contains the foreign key and table $T2$ contains the primary key. A column S is constructed by sorting $S1'$ and $S2'$. The prover checks S is a permutation of $S1' \cup S2'$ and that $S_i < S_{i+1}$ for all i . The join results, as depicted in the last four columns, are derived from the green-highlighted values in $GD1$ and $GD1'$.

Complexity of a join gate. Given two tables $T1$ and $T2$ with the number of records denoted by $T1_{num}$ and $T2_{num}$ respectively, we categorize records that contribute to the join predicate as $T1^{join}$ and $T2^{join}$, and those that do not contribute as $T1^{disjoin}$ and $T2^{disjoin}$. The counts of these records are denoted as $T1_{join_num}$, $T2_{join_num}$, $T1_{disjoin_num}$, and $T2_{disjoin_num}$ respectively. We omit the copy constraints in this analysis as they are lightweight. A permutation or range check gate, sized at X , implies that X corresponding constraints are applied across two columns, each populated with X values. The computation of constraints for a join operation encompasses five distinct categories:

- Two permutation check gates with sizes $T1_{num}$ and $T2_{num}$.
- Two range check constraints with lookup tables (referred to by Equations 1 and 3) in sizes $T1_{disjoin_num} + T2_{num}$ and $T2_{disjoin_num} + T1_{num}$ for the deduplication process.
- One range check constraint with lookup tables (referred to by Equations 1 and 3) in sizes $T1_{disjoin_num} + T2_{num}$ and $T2_{disjoin_num} + T1_{num}$ (in the worst case) for sorting the deduplicated versions.
- The number of equality check constraints (in the form $x - y = 0$) for checking if corresponding values satisfy the join predicate, with a maximum of either $T1_{disjoin}$ or $T2_{disjoin}$, for columns such as $J1$ and $J1'$ in the given figure.
- One range check constraint with lookup tables (referred to by Equations 1 and 3) in the size of $\max(T1_{disjoin}, T2_{disjoin})$ to ensure all records joined with $T2^{join}$ originate exclusively from $T2^{join}$.

4.5 Aggregation and Other Operations

Since aggregation operations are often applied together with the group-by operation, we describe how to implement them in conjunction with group-by. The SUM gate is implemented by establishing a column that holds intermediate, non-final values for each group-by bin, as described in Figure 5 and Example 4.2. To identify the starting and ending indices of each group-by bin, we can follow the method introduced in the Group-by section 4.3. Once we determine the indices of these boundary records for each group-by bin, we can employ a similar approach to implement the COUNT gate. With the SUM and COUNT values determined for each bin, the AVERAGE gate can be naturally realized through a division gate that processes these values.

Furthermore, the MAX and MIN gates are facilitated by a sorting mechanism. By arranging the values in ascending order, the smallest and largest values, corresponding to the MIN and MAX gates respectively, can be directly identified as the first and last values in the sorted list. Along with these functionalities, we have implemented additional aggregate functions such as Standard Deviation, Variance, and Median. Additionally, we have developed capabilities for string matching and concatenation by validating the equality of sub-strings in two strings using lookup tables.

For projection operation, we use selectors to project the desired columns by setting them to 1 for inclusion and 0 for exclusion. Each selector controls a multiplication gate, multiplying the column by 1 or 0 based on whether it is part of the projection. To ensure that the projection results include all the desired columns without revealing their

连接谓词。找到匹配项后，将来自 T2 的记录与来自 T1 的对应记录连接起来，形成一条合并记录。为了验证连接结果，必须建立两个关键属性：(1) 相等性验证：对于每个连接后的记录 r ，我们设置多项式 $r.attr1 - r.attr2 = 0$ ，其中 $attr1$ 和 $attr2$ 分别是来自 T1 和 T2 的用于连接的属性。(2) 来源验证：为了确保所有与 T1 连接的记录都仅来自 T2，我们使用 4.1 节中描述的查找表。

为了在不依赖主键-外键关系的情况下处理连接操作，我们使用类似的方法，通过对 $T1$ 和 $T2$ 中的记录进行两两比较来计算连接结果。然而，上述方法可能无法涵盖 $T1$ 和 $T2$ 中所有对连接谓词有贡献的记录。为了确保完整性，我们还使用相同的去重和排序机制证明了 $T1$ 与 $T2$ 不相交。这确保了所有相关记录都包含在连接操作中。

可扩展性。在处理大量连接操作时，数据库查询引擎通常会按顺序执行这些连接，一次连接两个表。这会导致不同的执行计划产生不同数量的连接结果（包括总连接结果和中间连接结果）。因此，应在电路设计阶段之前识别并应用旨在减少连接结果总数和中间连接结果数量的优化措施。

正确性。为确保连接门的正确性，我们验证以下属性：(1) 属性 1：置换完整性。集合 $T_1 \cup T_1$ 和 $T_2 \cup T_2$ 分别是原始表 T_1 和 T_2 的置换。证明方法与证明 P 是 P 的置换的方法类似。详情请参阅 4.1 节中的证明。(2) 属性 2：完备性。 T_1 和 T_2 包含所有对连接谓词有贡献的记录。假设集合 T1 缺少对连接谓词有贡献的记录 x ，即 x 不在 T1 中。我们使用 4.2 节中详述的元素包含性和排序性来证明 T1 与 T2 不相交。这意味着 x 与 T2 不重叠，表明 x 对连接谓词没有贡献。这与假设相矛盾，证明 T1 包含所有相关记录。类似地，T2 包含所有对连接谓词有贡献的记录。(3) 性质 3：互斥性。T1 和 T2 不包含对连接谓词没有贡献的记录。假设 T1 包含一条不满足连接条件的记录 y 。在连接过程中，与 y 连接的记录 r 将不满足约束 $r.attr1 - r.attr2 = 0$ ，这与假设相矛盾。因此，T1 和 T2 仅包含对连接谓词有贡献的记录。以上三个属性的正确性保证了连接门的正确性。

例 4.3。图 6 展示了 SQL 查询“SELECT T1.D1, T2.D2 FROM T1, T2 WHERE T1.D1 = T2.D1”。在接收到输入表 T1 和 T2 后，验证器会创建新表 T1 和 T2。上半部分包含用于连接 T1.D1 = T2.D1 的记录。列 G1 和 G2 中的绿色区域显示了相应的值。证明器通过构建列 S1 和 S2，去除重复项，并确保 S1 和 S2 中的每个值都存在于各自排序后的列 S1 和 S2 中，来验证 $GD1$ 和 $GD1$ 中的非贡献记录（灰色区域）不相交。证明 $GD1$ 中的非贡献记录（灰色区域）与 $GD1$ 中的贡献记录（绿色区域）不相交的过程也以类似的方式执行。

此步骤在示例中省略，并假设表 $T1$ 包含外键，表 $T2$ 包含主键。列 S 通过对 $S1$ 和 $S2$ 进行排序而构造。证明器检查 S 是否为 $S1 \cup S2$ 的排列，以及对于所有 i ， $S < S$ 。连接结果（如最后四列所示）源自 $GD1$ 和 $GD1$ 中绿色高亮显示的值。

连接门的复杂度。给定两个表 $T1$ 和 $T2$ ，记录数分别用 $T1$ 和 $T2$ 表示。我们将对连接谓词有贡献的记录分别记为 $T1$ 和 $T2$ ，将不参与连接谓词的记录分别记为 $T1$ 和 $T2$ 。这些记录的计数分别记为 $T1$ 、 $T2$ 、 $T1$ 和 $T2$ disjoin_num。由于复制约束较为简单，因此我们在此分析中省略了它们。大小为 X 的排列或范围检查门意味着在两个分别填充 X 个值的列上应用 X 个相应的约束。连接操作的约束计算包含五个不同的类别：

- 两个大小分别为 $T1$ 和 $T2$ 的置换校验门。
- 去重过程中，使用两个大小分别为 $T1+T2$ 和 $T2+T1$ 的查找表（如公式 1 和 3 所示）进行两个范围检查约束。

- 一个范围检查约束，使用查找表（如公式 1 和 3 中所述），大小为 $T1$ disjoin_num + $T2$ 和 $T2$ disjoin_num + $T1$ （在最坏情况下），用于对去重版本进行排序。
- 给定图中的列（例如 $J1$ 和 $J1$ ）的相等性检查约束（形式为 $x - y = 0$ ）的数量，用于检查对应值是否满足连接谓词，最大数量为 $T1$ 或 $T2$ 。

- 使用查找表（如公式 1 和 3 所示）进行一个范围检查约束，查找表的大小为 $\max(T1, T2)$ ，以确保所有记录与 T 连接 2 完全源自 T 2 。

4.5 聚合和其他操作

由于聚合操作通常与分组操作一起使用，因此我们在此介绍如何将它们与分组操作结合使用。求和门的实现方式是：为每个分组区间创建一个列，用于存储中间的非最终值，如图 5 和示例 4.2 所示。为了确定每个分组区间的起始索引和结束索引，我们可以采用分组操作 4.3 节中介绍的方法。一旦确定了每个分组区间边界记录的索引，我们就可以采用类似的方法来实现计数门。确定了每个区间的求和值和计数值后，就可以通过一个处理这些值的除法门自然地实现平均值门。

此外，MAX 和 MIN 门控功能由排序机制提供支持。通过按升序排列数值，最小值和最大值（分别对应于 MIN 和 MAX 门控）可以直接识别为排序列表中的第一个值和最后一个值。除了这些功能之外，我们还实现了其他聚合函数，例如标准差、方差和中位数。此外，我们还开发了字符串匹配和连接功能，通过使用查找表验证两个字符串中子字符串的相等性。

对于投影操作，我们使用选择器来投影所需的列，将其设置为 1 表示包含，设置为 0 表示排除。每个选择器控制一个乘法门，根据列是否属于投影范围，将该列乘以 1 或 0。这样可以确保投影结果包含所有所需的列，同时又不泄露它们的属性。

positional information, we employ a lookup table technique. This approach guarantees that the output preserves the required columns while maintaining the privacy of their original positions.

The set operations can be implemented using the methods described for the join gate. Set equality is handled by first sorting both tables and then comparing tuples at each index. Set disjointness is checked by sorting both tables and ensuring that any consecutive tuples R_i and R_{i+1} in the sorted list satisfy $R_i \leq R_{i+1}$. Set intersection is achieved as illustrated in Example 4.3 and Figure 6, where the join method is applied to extract common tuples between tables R and S . For set union, tuples that are common to both R and S (found via set equality) are first removed from R , and the remaining tuples are then concatenated to S .

We have covered the most common operations used in SQL queries. Other variations of these operations can be constructed using the methods introduced in this work, as long as they can be represented within the circuit framework.

4.6 Combining Gates

PoneglyphDB processes full SQL queries by combining customized gates for different operators as follows:

1. **Mapping Operations to Gates:** Each SQL query operation, such as sorting or joining, is represented by a corresponding gate. This gate executes the specific operation, ensuring accurate relationships between inputs and outputs.
2. **Predefined Execution Plan and Assembly:** The SQL query's predefined execution plan outlines the sequence and dependencies of operations, guiding the assembly of gates in sequence. Each gate's output serves as the input for the next, ensuring data flows correctly through the circuit according to the optimized plan.
3. **Combining Gates:** Multiple gates are combined to handle various operations as outlined in the execution plan. The gates are strategically assembled in sequence, with the output of one gate serving as the input for the next.

Since each operator (such as sorting or aggregation) is verified separately, proving each ensures the correctness of the entire query. However, even when all inputs appear in the output, as with sorting, there is a risk of data leakage from intermediate steps like comparisons. This is because intermediate steps, like comparing the relative order of data elements, may reveal unintended patterns or relationships. This is why we implement oblivious circuits: to ensure that no information about intermediate steps, such as comparisons or data order, is exposed during proof generation.

Correctness. Let $G_1, G_2, \dots, G_n$ be a sequence of gates processing a query. Assume G_1 receives correctly transformed input and operates correctly, yielding correct output O_1 . Assume for each G_i (where $1 \leq i \leq n$), the output O_i is correct and serves as input to G_{i+1} . Given each gate's correctness, G_{i+1} operates correctly on O_i to produce O_{i+1} . By induction, the final output O_n produced by the last gate G_n is correct.

5 EXPERIMENTAL RESULTS

We evaluate PoneglyphDB in terms of proving and verification time, memory usage, operator performance, proof size and scalability.

5.1 Experimental Setup

We implement PoneglyphDB's circuits and gates using Halo2, a state-of-the-art ZKP system [44]. The implementation of all Plonkish circuits for the queries is conducted using Rust. Our evaluation of PoneglyphDB focuses on a selected subset of the TPC-H benchmark [39], specifically targeting queries that are representative of common data analytics workloads.

We compare with ZKSQL [30], a state-of-the-art solution for using interactive ZKP in database systems. The interactivity in ZKSQL involves breaking down the computation into smaller sub-circuits to reduce the complexity of the overall circuit. These sub-circuits are then verified interactively, where the prover demonstrates the correctness of each sub-circuit step-by-step. The interaction ensures that the combined outputs of the sub-circuits correspond to the correct execution of the SQL query, allowing the verifier to check the correctness of the entire query process incrementally. However, this interactivity increases the communication and computational overhead, as each round of interaction requires multiple exchanges between the prover and verifier.

To align with existing research evaluations of ZKP-based databases and to conduct a fair comparison with ZKSQL, we have implemented the six TPC-H queries identified in their evaluation: Q1, Q3, Q5, Q8, Q9, and Q18.

In addition, we compare our system with Libra [40], a state-of-the-art non-interactive ZKP system that leverages the GKR protocol [20], which is also foundational to vSQL [47]. To the best of our knowledge, Libra is the most efficient publicly available system utilizing the GKR protocol. Since the high-level logic for implementing SQL operations in vSQL can be adapted across various ZKP systems, we use Libra's circuit structure to implement SQL operations based on the logic and optimizations introduced in vSQL. While Libra uses a fixed input structure, we adopt an alternative approach to avoid the need for relay gates for inputs that may not be needed immediately. That is, we split the circuit into multiple parts, ensuring that each part includes only the necessary inputs in its input layer.

In our experimentation, we adhere to the same query variables wherever applicable, such as the `orderdate` filter, to maintain consistency and relevance in our results. For query Q9, similar to ZKSQL's approach, we exclude string pattern-matching predicates from our evaluation. We converted all floating point operations to 64-bit integer ones in our experiments similar to ZKSQL.

Our experimental setup quantifies the database scale by the size of the central fact table, `lineitem`, and scales the dimension tables proportionally, as described in the TPC-H benchmark specifications. We report results across three database sizes—60k Rows, 120k Rows, and 240k Rows—with the `lineitem` table containing 60k, 120k, and 240k rows, respectively. These varying sizes provide insights into the scalability and practicality of PoneglyphDB in handling verifiable databases across different volumes of data. Unless otherwise mentioned, our experiments run on 60k Rows.

Our experiments are conducted on Chameleon Cloud [27] using a Skylake node, equipped with two Intel Xeon Skylake CPUs running at 2.60 GHz, 192 GB of RAM, and 10 Gigabit Ethernet connectivity.

5.2 Setup

PoneglyphDB eliminates the need for a trusted setup process. Instead, PoneglyphDB utilizes public parameters. These parameters are essential for both constructing and verifying proofs and are known to all

为了存储位置信息，我们采用了一种查找表技术。这种方法既能保证输出结果保留所需的列，又能保护原始位置信息的隐私性。

集合运算可以使用连接门中描述的方法来实现。集合相等性检查首先对两个表进行排序，然后比较每个索引处的元组。集合不相交性检查通过对两个表进行排序，并确保排序列表中任何连续的元组 R 和 R 满足 $R \leq R$ 来实现。集合交集的实现方式如示例 4.3 和图 6 所示，其中应用连接方法提取表 R 和 S 之间的公共元组。对于集合并集，首先从 R 中移除 R 和 S 共有的元组（通过集合相等性找到），然后将剩余的元组连接到 S 。

我们已经介绍了 SQL 查询中最常用的操作。只要能在电路框架内表示，就可以使用本文介绍的方法构建这些操作的其他变体。

4.6 组合门

PoneglyphDB 通过组合针对不同运算符的自定义门来处理完整的 SQL 查询，如下所示：

1. 将操作映射到门：每个 SQL 查询操作（例如排序或连接）都由一个对应的门表示。该门执行特定的操作，确保输入和输出之间准确的对应关系。
2. 预定义执行计划和组装：SQL 查询的预定义执行计划概述了操作的顺序和依赖关系，指导逻辑门按顺序组装。每个逻辑门的输出作为下一个逻辑门的输入，确保数据按照优化后的计划正确地在电路中流动。
3. 逻辑门组合：多个逻辑门组合在一起，以处理执行计划中概述的各种操作。这些逻辑门按顺序策略性地组装，一个逻辑门的输出作为下一个逻辑门的输入。

由于每个运算符（例如排序或聚合）都单独验证，因此证明每个运算符的正确性即可确保整个查询的正确性。然而，即使所有输入都出现在输出中（例如排序），仍然存在数据泄露的风险，因为中间步骤（例如比较）可能会泄露数据。这是因为中间步骤（例如比较数据元素的相对顺序）可能会揭示意料之外的模式或关系。因此，我们实现了不经意电路：以确保在生成证明的过程中不会泄露任何关于中间步骤（例如比较或数据顺序）的信息。

正确性。设 $G, G, \dots, G$ 为处理查询的逻辑门序列。假设 G 接收正确转换的输入并正确操作，产生正确的输出 O 。假设对于每个 G （其中 $1 \leq i \leq n$ ），输出 O 都是正确的，并作为 G 的输入。鉴于每个逻辑门的正确性， G 对 O 进行正确操作，产生 O 。通过归纳法，最后一个逻辑门 G 产生的最终输出 O 也是正确的。

5 实验结果

我们从证明和验证时间、内存使用情况、运算符性能、证明大小和可扩展性等方面对 PoneglyphDB 进行评估。

5.1 实验装置

我们使用 Halo2（一种先进的零知识证明系统 [44]）实现了 PoneglyphDB 的电路和门。所有用于查询的 Plonkish 电路均使用 Rust 实现。我们对 PoneglyphDB 的评估侧重于 TPC-H 基准测试 [39] 的一个选定子集，专门针对能够代表常见数据分析工作负载的查询。

我们将本文方法与 ZKSQL [30] 进行比较，ZKSQL 是目前用于数据库系统中交互式零知识证明 (ZKP) 的一种先进解决方案。ZKSQL 的交互性体现在将计算分解为更小的子电路，从而降低整个电路的复杂度。这些子电路随后会进行交互式验证，证明者会逐步证明每个子电路的正确性。这种交互确保了子电路的组合输出与 SQL 查询的正确执行相对应，从而使验证者能够逐步检查整个查询过程的正确性。然而，这种交互性会增加通信和计算开销，因为每一轮交互都需要证明者和验证者之间进行多次交换。

为了与现有的基于 ZKP 的数据库的研究评估保持一致，并与 ZKSQL 进行公平的比较，我们实现了他们在评估中确定的六个 TPC-H 查询：Q1、Q3、Q5、Q8、Q9 和 Q18。

此外，我们将我们的系统与 Libra [40] 进行比较。Libra 是一个先进的非交互式零知识证明 (ZKP) 系统，它利用了 GKR 协议 [20]，而 GKR 协议也是 vSQL [47] 的基础。据我们所知，Libra 是目前公开可用的最高效的 GKR 协议系统。由于 vSQL 中用于实现 SQL 操作的高级逻辑可以适用于各种 ZKP 系统，因此我们利用 Libra 的电路结构，基于 vSQL 中引入的逻辑和优化来实现 SQL 操作。Libra 使用固定的输入结构，而我们采用了一种替代方法，以避免为可能不需要立即使用的输入使用中继门。也就是说，我们将电路分成多个部分，确保每个部分的输入层只包含必要的输入。

在我们的实验中，我们尽可能使用相同的查询变量，例如订单日期筛选器，以确保结果的一致性和相关性。对于查询 Q9，与 ZKSQL 的方法类似，我们从评估中排除了字符串模式匹配谓词。与 ZKSQL 类似，我们在实验中将所有浮点运算转换为 64 位整数运算。

我们的实验设置通过中心事实表（行项目表）的大小来量化数据库规模，并按比例缩放维度表，如 TPC-H 基准测试规范中所述。我们报告了三种数据库规模（6 万行、12 万行和 24 万行）的结果，其中行项目表分别包含 6 万行、12 万行和 24 万行。这些不同的规模有助于深入了解 PoneglyphDB 在处理不同数据量可验证数据库时的可扩展性和实用性。除非另有说明，我们的实验均在 6 万行数据上运行。

我们的实验是在 Chameleon Cloud [27] 上进行的，使用 Skylake 节点，该节点配备两个运行频率为 2.60 GHz 的 Intel Xeon Skylake CPU、192 GB RAM 和 10 千兆以太网连接。

5.2 设置

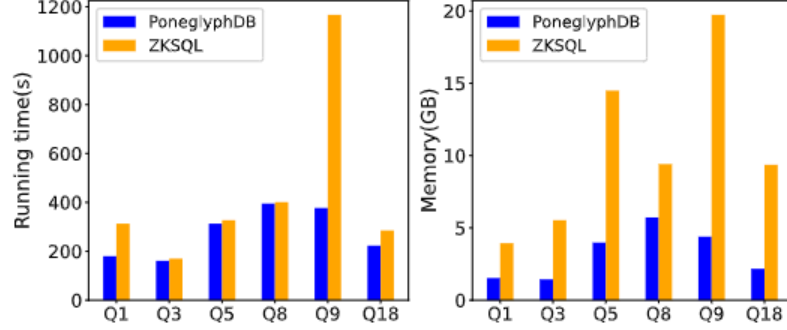
PoneglyphDB 无需可信的设置过程。相反，PoneglyphDB 使用公共参数。这些参数对于构建和验证证明都至关重要，并且所有人都知道。

Table 2: Running time (in seconds) for generating public parameters with different maximal number of rows in Plonkish circuits.

Maximal number of rows	2^{15}	2^{16}	2^{17}	2^{18}
Running time (s)	104	221	410	832

Table 3: Running time (in seconds) of database commitment over data of increasing sizes.

The size of the database	60k Rows	120k Rows	240k Rows
Running time (s)	2.89	5.53	10.94

**Figure 7: Running time (left figure) and memory usage (right figure) for generating SQL queries proofs.**

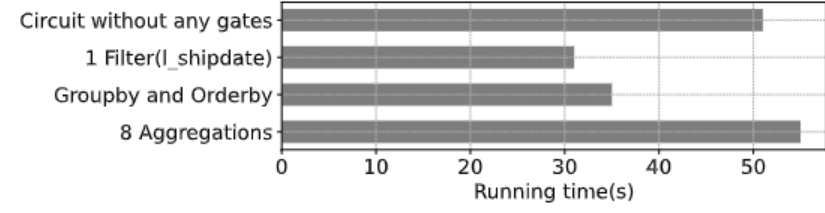
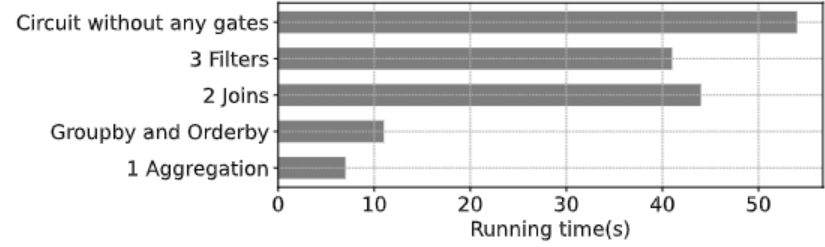
parties involved—the prover and the verifier alike. Importantly, these parameters are not confidential and require no secrecy.

Table 2 details the running time associated with generating these public parameters. It’s worth noting that the generation of these parameters is a one-time process; once created, they can be stored and reused indefinitely. The versatility of these parameters allows for their application across various circuits, provided the number of rows in the circuit does not surpass the maximum capacity defined by the public parameters. Consequently, the time spent generating these parameters is not considered part of the cost associated with generating SQL query proofs in this work.

Running time of database commitment. The proof generation for a fixed database commitment can be done once and be reused for SQL queries that are applied on the database. Table 3 shows the running time of committing to the 8 TPC-H tables.

5.3 Benchmarking with ZKSQL

We compare the running time of generating proofs for six SQL queries in PoneglyphDB with that of ZKSQL. The results (the left figure of Figure 7), reveal that PoneglyphDB—although a non-interactive ZKP solution—achieves performance that is similar to the interactive ZKP solution ZKSQL for most queries. In fact, PoneglyphDB outperforms ZKSQL significantly—by at least up to 40%—for queries Q1 and Q9. This difference is attributed to the relatively fewer range check (or filtering) and sort operations required in Q1 and Q9. PoneglyphDB utilizes arithmetic circuits for handling range checks and sorting operations. Despite the use of lookup tables to optimize the degrees of polynomial constraints and reduce the circuit size, arithmetic circuits can become more complex than boolean circuits, which ZKSQL employs for filtering and sorting operations, especially as the range of data increases. Nevertheless, PoneglyphDB exhibits enhanced performance in join operations, which necessitate arithmetic expressions to represent polynomial constraints accurately. Figure 10 (right) shows the memory usage for generating proofs for the six SQL queries in PoneglyphDB and ZKSQL. PoneglyphDB uses significantly less memory, ranging from 23% to 60% of ZKSQL’s usage.

**Figure 8: PoneglyphDB’s performance breakdown of different proof generation steps for Q1.****Figure 9: PoneglyphDB’s performance breakdown of different proof generation steps for Q3.**

5.4 Benchmarking with Libra

Since Libra is a non-interactive ZKP system, we benchmarked PoneglyphDB against Libra in terms of three critical factors: proving time, verification time, and proof size. As shown in Table 4, Libra requires more proving time than PoneglyphDB. In Libra, complex operations such as sorting require a large number of basic gates, with each gate limited to two inputs, which increases both the circuit depth and size. For comparison operations in SQL queries, decimal values are represented using full 64-bit binary representations in Libra. Logical operations on these 64-bit binary numbers necessitate circuits that handle each bit individually, including managing carry bits across the entire bit width. This bitwise processing, along with the overhead of transforming binary values to decimal for subsequent arithmetic operations, results in significantly larger circuits. This increased circuit size leads to longer proving times. In contrast, PoneglyphDB optimizes the handling of decimal values by segmenting them into 8-bit chunks and leveraging lookup tables to efficiently validate and perform operations on each segment. The larger circuit size in Libra not only increases the proving time but also leads to longer verification times and larger proof sizes, as shown in Table 4 for queries Q1, Q3, and Q5.

5.5 Operation Performance

To enhance our understanding of PoneglyphDB’s performance, we assess the overheads associated with the steps involved in proof generation. Figures 8 and 9 show a breakdown of the execution time to generate proofs for queries Q1 and Q3, respectively. We selected these two queries for our performance evaluation because they encompass a comprehensive range of SQL operations, including multiple aggregations, joins, group-by, sort, and filtering functions.

The proof generation process begins with the construction of a comprehensive circuit, encapsulating all facets of witness generation; this preliminary phase is described as a “circuit without any gates”. Following this, the procedure advances to the integration of polynomial constraints—or gates—that correspond to the SQL operations. These operations include filtering, grouping by, ordering, and performing eight aggregations for query Q1, as well as applying three filters, executing two joins, a group-by, an order-by, and an aggregation for query Q3.

The results show that the initial step takes over 50 seconds, attributable to the fixed overheads determined by the chosen public parameter;

表 2: Plonkish 电路中不同最大行数下生成公共参数的运行时间 (秒)。

最大行数	2222	运行时间 (秒)			
	104	221	410	832	

表 3: 数据库提交对不断增大的数据的运行时间 (以秒为单位)。

数据库大小	60k ROWS	120k ROWS	240k ROWS	
运行时间 (秒)	2.89	5.53	10.94	

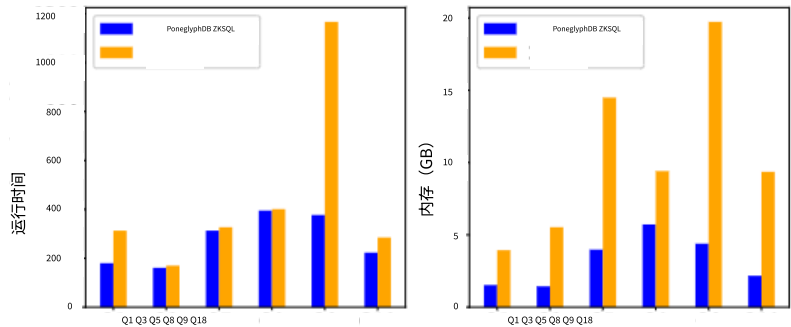


图 7: 生成 SQL 查询证明的运行时间 (左图) 和内存使用情况 (右图)。

相关各方——证明者和验证者——都参与其中。重要的是，这些参数并非机密信息，无需保密。

表 2 详细列出了生成这些公共参数所需的运行时间。值得注意的是，这些参数的生成是一次性过程；一旦创建，它们就可以存储并无限期地重复使用。这些参数的通用性使其可以应用于各种电路，前提是电路中的行数不超过公共参数定义的最大容量。因此，生成这些参数所花费的时间不计入本文中生成 SQL 查询证明的成本。

数据库提交的运行时间。证明生成

对于固定的数据库，只需执行一次提交操作，即可将其重复用于应用于该数据库的 SQL 查询。表 3 显示了提交到 8 个 TPC-H 表的运行时间。

5.3 使用 ZKSQL 进行基准测试

我们将 PoneglyphDB 和 ZKSQL 生成六个 SQL 查询证明的运行时间进行了比较。结果 (图 7 左图) 表明，PoneglyphDB 虽然是交互式 ZKP 解决方案，但对于大多数查询，其性能与交互式 ZKP 解决方案 ZKSQL 相当。事实上，对于查询 Q1 和 Q9，PoneglyphDB 的性能显著优于 ZKSQL，至少提升了 40%。这种差异归因于 Q1 和 Q9 中所需的范围检查 (或过滤) 和排序操作相对较少。PoneglyphDB 使用算术电路来处理范围检查和排序操作。尽管使用了查找表来优化多项式约束的次数并减小电路规模，但算术电路的复杂度仍然可能高于 ZKSQL 用于过滤和排序操作的布尔电路，尤其是在数据范围增大时。然而，PoneglyphDB 在连接操作方面表现出更优的性能，因为连接操作需要使用算术表达式来精确表示多项式约束。图 10 (右) 展示了 PoneglyphDB 和 ZKSQL 中生成六个 SQL 查询证明的内存使用情况。PoneglyphDB 的内存使用量显著低于 ZKSQL，仅为 ZKSQL 的 23% 到 60%。

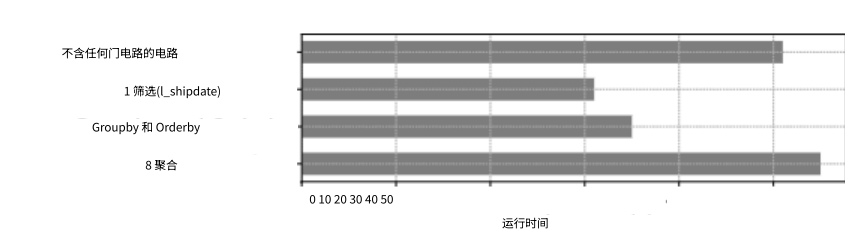


图 8: PoneglyphDB 针对 Q1 的不同证明生成步骤的性能细分。

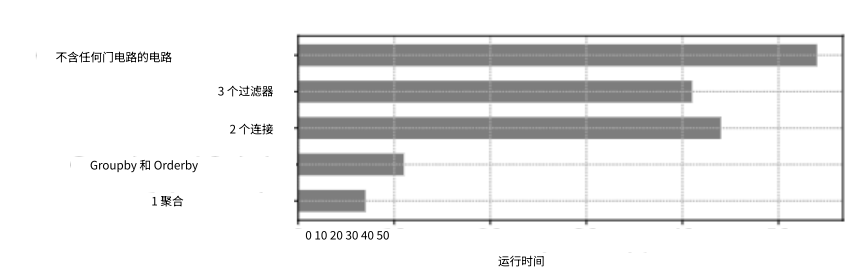


图 9: PoneglyphDB 第三季度不同证明生成步骤的性能细分。

5.4 使用 Libra 进行基准测试

由于 Libra 是一个非交互式零知识证明 (ZKP) 系统，我们从三个关键因素方面对 PoneglyphDB 和 Libra 进行了基准测试：证明时间、验证时间和证明规模。如表 4 所示，Libra 的证明时间比 PoneglyphDB 更长。在 Libra 中，诸如排序之类的复杂操作需要大量的基本门电路，而每个门电路的输入仅限于两个，这增加了电路的深度和规模。对于 SQL 查询中的比较操作，Libra 使用完整的 64 位二进制表示来表示十进制值。对这些 64 位二进制数进行逻辑运算需要电路来单独处理每一位，包括管理整个位宽上的进位。这种按位处理，以及将二进制值转换为十进制以进行后续算术运算的开销，导致电路规模显著增大。电路规模的增大导致了更长的证明时间。相比之下，PoneglyphDB 通过将十进制值分割成 8 位块，并利用查找表高效地验证每个块并执行操作，从而优化了十进制值的处理。Libra 中更大的电路规模不仅增加了证明时间，还导致更长的验证时间和更大的证明文件大小，如表 4 中查询 Q1、Q3 和 Q5 所示。

5.5 运营绩效

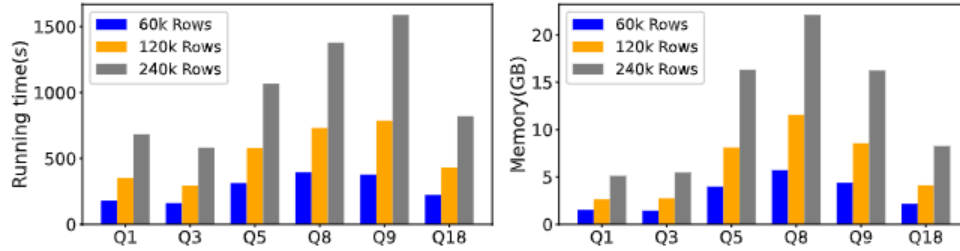
为了更深入地了解 PoneglyphDB 的性能，我们评估了证明生成过程中各个步骤的开销。图 8 和图 9 分别展示了查询 Q1 和 Q3 生成证明的执行时间细分。我们选择这两个查询进行性能评估，是因为它们涵盖了广泛的 SQL 操作，包括多种聚合、连接、分组、排序和筛选功能。

证明生成过程始于构建一个包含所有见证生成环节的综合电路；这一初步阶段被称为“无门电路”。随后，该过程推进到集成与 SQL 操作对应的多项式约束 (或称“门”)。这些操作包括对查询 Q1 进行过滤、分组、排序和八次聚合，以及对查询 Q3 应用三个过滤器、执行两次连接、一次分组、一次排序和一次聚合。

结果显示，初始步骤耗时超过 50 秒，这是由于所选公共参数决定的固定开销造成的；

Table 4: Benchmarking against Libra in terms of proving time, verification time and proof size.

		Proving time (in seconds)	Verification time (in seconds)	Proof size (in kilobytes)
Libra	Q1	812	1.290	435.8
	Q3	997	1.212	411.4
	Q5	1021	1.227	413.9
PoneglyphDB	Q1	180	0.617	8.6
	Q3	161	0.725	24.7
	Q5	313	0.739	29.6

**Figure 10: Proof generation time and memory usage over data of increasing sizes.**

a larger public parameter size increases this initial step overhead. The significant overhead in proof generation, notably from the aggregations in Q1 and the filters and joins in Q3, can be attributed to the extensive computational resources required. Aggregation operations, for instance, necessitate the collation and computation across sizable datasets to yield a singular summary outcome. This task demands multiple iterations of data processing and polynomial constraints verification within the circuit, thereby amplifying its complexity and extending the duration needed for proof generation. Similarly, filters and joins in Q3 are computationally intensive, as filtering requires checking each record against conditions, while joins involve aligning records from different tables based on join keys.

5.6 Scalability

We evaluate the scalability of PoneglyphDB by generating proofs with larger workloads. We evaluate with the workload of TPC-H's lineitem table at 120k and 240k rows as the lineitem table dominates the complexity of the SQL queries. As depicted in Figure 10, the running time and memory consumption increases with the increase in the size of the dataset. Specifically, the running times for the six SQL queries—Q1, Q3, Q5, Q8, Q9, and Q18—exhibit a gradual increase as the database size increases from 60k to 240k Rows.

The running time for query Q1 starts at 180 seconds for 60k rows and increases to 683 seconds for the 240k row dataset, indicating a proportional relationship between dataset size and observed performance. This is because the size of our circuits linearly grows with the size of inputs and all the polynomial constraints enforced on the circuits have low degrees. This validates our design goal of maintaining low-degree polynomials in PoneglyphDB's circuits. Memory usage shows a similar pattern, with the memory footprint for query Q1 starting at 1.53 GB for 60k rows and increasing to 5.12 GB for 240k rows.

6 Related Work

There is substantial research into verifiable SQL querying, employing a variety of techniques that ensure the integrity and security of query results. These methods can be broadly categorized into three groups: Authenticated Data Structures (ADS) [38], Trusted Execution Environments (TEE) [35], and Cryptographic Proof Techniques [17].

ADS-based methods use asymmetric cryptography to authenticate data, requiring extra memory to maintain authenticated data structures for SQL query verification [4, 33, 34, 41, 50]. While secure, these methods are generally limited to specific computational tasks.

TEE-based approaches, exemplified by [3, 4, 37, 51, 52], secure SQL query results through computations performed within trusted hardware environments. Basic TEE implementations might expose sensitive data through program traces. Integrating TEE with Oblivious Random Access Machine (ORAM), as in [23], can obscure such traces but at the cost of additional computation time, highlighted in [2, 29].

Cryptographic proof techniques, such as zk-SNARKs [6] and zk-STARKs [5], enable entities to verify the correctness of computations without revealing underlying data. These techniques have found widespread application across various domains, including blockchain off-chain computations and privacy-preserving machine learning [24, 25, 31, 53]. These techniques ensure high levels of security but are resource-intensive, requiring substantial memory to manage numerous intermediate values and considerable time to create proofs.

Prior cryptographic proof systems, such as IntegriDB [49] and vSQL [47], employ cryptographic verifiable computation to validate a wide range of SQL queries. While IntegriDB and vSQL ensure data integrity, they operate in an outsourcing model and do not inherently provide zero-knowledge properties. An extension of vSQL, referred to as vSQL+ [48], introduces ZKP; however, it lacks support for ad-hoc queries and does not thoroughly address practical efficiency or the translation of arbitrary SQL statements into cryptographic protocols necessary for such guarantees. Notably, vSQL and vSQL+ are based on public-coin protocols [21, 22], which can be transformed into non-interactive ZKP systems using the Fiat-Shamir heuristic [16]. ZKSQL [30] reduces the proving cost by dividing the entire circuit into smaller sub-circuits to reduce the size of the overall circuit. This approach supports ad-hoc queries and maintains zero-knowledge properties, but it shares the common limitations of interactive ZKP. ZKSQL is based on designated-verifier protocols [22], where the Fiat-Shamir heuristic cannot generally be applied to transform the protocol into a non-interactive proof.

Our system, PoneglyphDB, generates non-interactive ZKP using recursive proof composition [7, 9, 11, 28]. It enhances proof generation performance by optimizing arithmetic circuits.

7 Conclusion

We introduce PoneglyphDB, a non-interactive ZKP-based database designed for efficient confidentiality and provability. PoneglyphDB optimizes proof generation through recursive methods and tailored designs for SQL queries, showing competitive or superior performance in TPC-H benchmarks compared to existing solutions.

8 Acknowledgments

This research is partly supported by the NSF under grants CNS1815212 and SaTC-2245372.

References

- [1] Rakesh Agrawal, Jerry Kiernan, Ramakrishnan Srikant, and Yirong Xu. 2002. Hippocratic databases. In *VLDB'02: Proceedings of the 28th International Conference on Very Large Databases*. Elsevier, 143–154.
- [2] AKM Mubashwir Alam, Justin Boyce, and Keke Chen. 2023. SGX-MR-Prot: Efficient and Developer-Friendly Access-Pattern Protection in Trusted Execution

表 4：与 Libra 的比较，包括证明时间、验证时间和证明大小。

		证明时间 (以秒为单位)	验证时间 (以秒为单位)	样例尺寸 (单位：千字节)
天秤座	Q1	812	1.290	435.8
	Q3	997	1.212	411.4
	Q5	1021	1.227	413.9
历史正文数据库	Q1	180	0.617	8.6
	Q3	161	0.725	24.7
	Q5	313	0.739	29.6

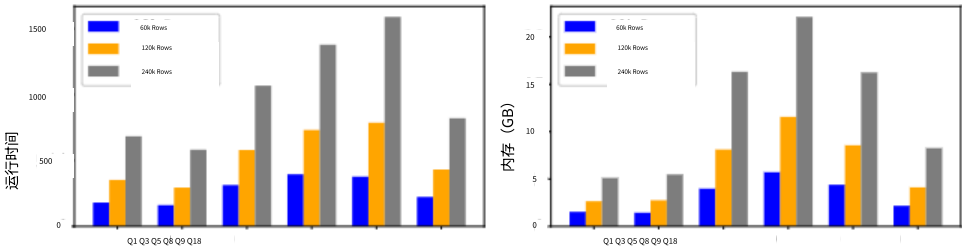


图 10：随着数据规模的增加，证明生成时间和内存使用情况。

更大的公共参数规模会增加初始步骤的开销。证明生成过程中显著的开销，特别是来自问题 1 中的聚合操作以及问题 3 中的过滤和连接操作的开销，可归因于所需的大量计算资源。例如，聚合操作需要对庞大的数据集进行整理和计算，才能得出单一的汇总结果。这项任务需要在电路中进行多次数据处理和多项式约束验证，从而增加了其复杂性并延长了证明生成所需的时间。类似地，问题 3 中的过滤和连接操作也需要大量的计算，因为过滤需要检查每条记录是否符合条件，而连接则涉及根据连接键将来自不同表的记录对齐。

5.6 可扩展性

我们通过生成更大工作负载的证明来评估 PoneglyphDB 的可扩展性。我们使用 TPC-H 的 lineitem 表（12 万行和 24 万行）的工作负载进行评估，因为 lineitem 表是 SQL 查询复杂度的主要来源。如图 10 所示，运行时间和内存消耗随着数据集大小的增加而增加。具体而言，六个 SQL 查询（Q1、Q3、Q5、Q8、Q9 和 Q18）的运行时间随着数据库大小从 6 万行增加到 24 万行而逐渐增加。

查询 Q1 的运行时间在 6 万行数据集上为 180 秒，在 24 万行数据集上增加到 683 秒，表明数据集大小与性能呈正比关系。这是因为我们电路的大小随输入大小线性增长，并且所有施加在电路上的多项式约束的次数都很低。这验证了我们在 PoneglyphDB 电路中保持低次数多项式的设计目标。内存使用情况也呈现类似的模式，查询 Q1 的内存占用在 6 万行数据集上为 1.53 GB，在 24 万行数据集上增加到 5.12 GB。

6 相关工作

针对可验证 SQL 查询已有大量研究，采用了多种技术来确保查询结果的完整性和安全性。这些方法大致可分为三类：认证数据结构 (ADS) [38]、可信执行环境 (TEE) [35] 和密码学证明技术 [17]。

基于 ADS 的方法使用非对称加密技术来验证数据，需要额外的内存来维护用于 SQL 查询验证的已验证数据结构[4, 33, 34, 41, 50]。虽然这些方法安全可靠，但通常仅限于特定的计算任务。

基于可信执行环境 (TEE) 的方法，例如 [3, 4, 37, 51, 52]，通过在可信硬件环境中执行计算来保护 SQL 查询结果。基本的 TEE 实现可能会通过程序跟踪暴露敏感数据。将 TEE 与不经意随机存取机 (ORAM) 集成（如 [23] 所示）可以隐藏此类跟踪，但代价是需要额外的计算时间，这一点在 [2, 29] 中有所提及。

诸如 zk-SNARKs [6] 和 zkSTARKs [5] 等密码学证明技术，使得实体能够在不泄露底层数据的情况下验证计算的正确性。这些技术已在包括区块链链下计算和隐私保护机器学习在内的各个领域得到广泛应用 [24, 25, 31, 53]。这些技术能够确保高安全性，但资源消耗巨大，需要大量内存来管理众多中间值，并且需要相当长的时间来生成证明。先前的加密证明系统，例如 IntegriDB [49] 和 vSQL [47]，采用加密可验证计算来验证各种 SQL 查询。虽然 IntegriDB 和 vSQL 能够确保数据完整性，但它们采用外包模式，本身并不具备零知识证明 (ZKP) 特性。vSQL 的一个扩展版本 vSQL+ [48] 引入了 ZKP；然而，它缺乏对即席查询的支持，也没有充分解决实际效率问题，或者如何将任意 SQL 语句转换为此类保证所需的加密协议。值得注意的是，vSQL 和 vSQL+ 基于公币协议 [21, 22]，这些协议可以使用 Fiat-Shamir 启发式算法 [16] 转换为非交互式 ZKP 系统。ZKSQL [30] 通过将整个电路划分为更小的子电路来降低证明成本，从而减小整个电路的规模。这种方法支持临时查询并保持零知识属性，但它也存在交互式零知识证明的常见局限性。ZKSQL 基于指定验证者协议 [22]，其中 Fiat-Shamir 启发式方法通常不能用于将该协议转换为非交互式证明。我们的系统 PoneglyphDB 使用递归证明组合生成非交互式零知识证明 [7, 9, 11, 28]。它通过优化算术电路来提高证明生成性能。

7 结论

我们推出了 PoneglyphDB，这是一个基于零知识证明（ZKP）的非交互式数据库，旨在实现高效的保密性和可证明性。PoneglyphDB 通过递归方法和针对 SQL 查询的定制设计来优化证明生成，在 TPC-H 基准测试中展现出与现有解决方案相当甚至更优的性能。

8 致谢

本研究部分由美国国家科学基金会资助，资助编号为 CNS1815212 和 SaTC-2245372。

参考

[1] Rakesh Agrawal, Jerry Kiernan, Ramakrishnan Srikant 和 Yirong Xu. 2002 年。Hippocratic 数据库。载于 VLDB'02：第 28 届国际超大型数据库会议论文集。Elsevier 出版社，143-154 页。
[2] AKM Mubashwir Alam, Justin Boyce 和 Keke Chen. 2023. SGX-MR-Prot：可信执行中高效且对开发者友好的访问模式保护

- Environments. In *2023 IEEE 43rd International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 1029–1032.
- [3] Arvind Arasu, Ken Eguro, Raghav Kaushik, Donald Kossmann, Pingfan Meng, Vineet Pandey, and Ravi Ramamurthy. 2017. Concerto: A high concurrency key-value store with integrity. In *Proceedings of the 2017 ACM International Conference on Management of Data*. 251–266.
 - [4] Sumeet Bajaj and Radu Sion. 2013. CorrectDB: SQL engine with practical query authentication. *Proceedings of the VLDB Endowment* 6, 7 (2013), 529–540.
 - [5] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. 2019. Scalable zero knowledge with no trusted setup. In *Advances in Cryptology—CRYPTO 2019: 39th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18–22, 2019, Proceedings, Part III* 39. Springer, 701–732.
 - [6] Eli Ben-Sasson, Alessandro Chiesa, Daniel Genkin, Eran Tromer, and Madars Virza. 2013. SNARKs for C: Verifying program executions succinctly and in zero knowledge. In *Annual cryptography conference*. Springer, 90–108.
 - [7] Eli Ben-Sasson, Alessandro Chiesa, Eran Tromer, and Madars Virza. 2017. Scalable zero knowledge via cycles of elliptic curves. *Algorithmica* 79, 4 (2017), 1102–1160.
 - [8] Jonathan Bootle, Andrea Cerulli, Pyrros Chaidos, Jens Groth, and Christophe Petit. 2016. Efficient zero-knowledge arguments for arithmetic circuits in the discrete log setting. In *Advances in Cryptology—EUROCRYPT 2016: 35th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Vienna, Austria, May 8–12, 2016, Proceedings, Part II* 35. Springer, 327–357.
 - [9] Sean Bowe, Jack Grigg, and Daira Hopwood. 2019. Recursive proof composition without a trusted setup. *Cryptology ePrint Archive* (2019).
 - [10] Benedikt Bünz, Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, and Greg Maxwell. 2018. Bulletproofs: Short proofs for confidential transactions and more. In *2018 IEEE symposium on security and privacy (SP)*. IEEE, 315–334.
 - [11] Benedikt Bünz, Alessandro Chiesa, Pratyush Mishra, and Nicholas Spooner. 2020. Recursive proof composition from accumulation schemes. In *Theory of Cryptography: 18th International Conference, TCC 2020, Durham, NC, USA, November 16–19, 2020, Proceedings, Part II* 18. Springer, 1–18.
 - [12] Ji-Won Byun and Ninghui Li. 2008. Purpose based access control for privacy protection in relational database systems. *The VLDB Journal* 17 (2008), 603–619.
 - [13] Alessandro Chiesa, Yuncong Hu, Mary Maller, Pratyush Mishra, Noah Vesely, and Nicholas Ward. 2020. Marlin: Preprocessing zkSNARKs with universal and updatable SRS. In *Advances in Cryptology—EUROCRYPT 2020: 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, May 10–14, 2020, Proceedings, Part I* 39. Springer, 738–768.
 - [14] Tal Derei, Benjamin Aulenbach, Victor Carolino, Caleb Geren, Michael Kaufman, Jonathan Klein, Rishad Islam Shanto, and Henry F Korth. 2023. Scaling Zero-Knowledge to Verifiable Databases. In *Proceedings of the 1st Workshop on Verifiable Database Systems*. 1–9.
 - [15] Cynthia Dwork. 2006. Differential privacy. In *International colloquium on automata, languages, and programming*. Springer, 1–12.
 - [16] Amos Fiat and Adi Shamir. 1986. How to prove yourself: Practical solutions to identification and signature problems. In *Conference on the theory and application of cryptographic techniques*. Springer, 186–194.
 - [17] Uriel Fiege, Amos Fiat, and Adi Shamir. 1987. Zero knowledge proofs of identity. In *Proceedings of the nineteenth annual ACM symposium on Theory of computing*. 210–217.
 - [18] Georg Fuchsbaue, Eike Kiltz, and Julian Loss. 2018. The algebraic group model and its applications. In *Advances in Cryptology—CRYPTO 2018: 38th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 19–23, 2018, Proceedings, Part II* 38. Springer, 33–62.
 - [19] Ariel Gabizon and Zachary J Williamson. 2020. plookup: A simplified polynomial protocol for lookup tables. *Cryptology ePrint Archive* (2020).
 - [20] Shafi Goldwasser, Yael Tauman Kalai, and Guy N Rothblum. 2015. Delegating computation: interactive proofs for muggles. *Journal of the ACM (JACM)* 62, 4 (2015), 1–64.
 - [21] S. Goldwasser, S. Micali, and C. Rackoff. 1985. The Knowledge Complexity of Interactive Proof-Systems. In *Proceedings of the 17th Annual ACM Symposium on Theory of Computing, May 6–8, 1985, Providence, Rhode Island, USA*.
 - [22] Shafi Goldwasser and Michael Sipser. 1986. Private coins versus public coins in interactive proof systems. In *Proceedings of the eighteenth annual ACM symposium on Theory of computing*. 59–68.
 - [23] Michael T Goodrich, Michael Mitzenmacher, Olga Ohrimenko, and Roberto Tamassia. 2012. Privacy-preserving group data access via stateless oblivious RAM simulation. In *Proceedings of the twenty-third annual ACM-SIAM symposium on Discrete Algorithms*. SIAM, 157–167.
 - [24] Binbin Gu and Faisal Nawab. 2024. zk-oracle: Trusted off-chain compute and storage for decentralized applications. *Distributed and Parallel Databases* (2024), 1–24.
 - [25] Binbin Gu, Abhishek Singh, Yanan Zhou, Juncheng Fang, and Faisal Nawab. 2023. ML on Chain: The Case and Taxonomy of Machine Learning on Blockchain. In *2023 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*. IEEE, 1–18.
 - [26] Noah Johnson, Joseph P Near, and Dawn Song. 2018. Towards practical differential privacy for SQL queries. *Proceedings of the VLDB Endowment* 11, 5 (2018), 526–539.
 - [27] Kate Keahey, Jason Anderson, Zhuo Zhen, Pierre Riteau, Paul Ruth, Dan Stanzione, Mert Cevik, Jacob Colleran, Haryadi S. Gunawi, Cody Hammock, Joe Mambretti, Alexander Barnes, François Halbach, Alex Rocha, and Joe Stubbs. 2020. Lessons Learned from the Chameleon Testbed. In *Proceedings of the 2020 USENIX Annual Technical Conference (USENIX ATC '20)*. USENIX Association.
 - [28] Abhiram Kothapalli, Srinath Setty, and Ioanna Tzialla. 2022. Nova: Recursive zero-knowledge arguments from folding schemes. In *Annual International Cryptology Conference*. Springer, 359–388.
 - [29] Duc V Le, Lizzy Tengana Hurtado, Adil Ahmad, Mohsen Minaei, Byoungyoung Lee, and Aniket Kate. 2020. A tale of two trees: one writes, and other reads. *Proceedings on Privacy Enhancing Technologies* (2020).
 - [30] Xiling Li, Chenkai Weng, Yongxin Xu, Xiao Wang, and Jennie Rogers. 2023. ZKSQL: Verifiable and Efficient Query Evaluation with Zero-Knowledge Proofs. *Proceedings of the VLDB Endowment* 16, 8 (2023), 1804–1816.
 - [31] Qi Lin, Binbin Gu, and Faisal Nawab. 2024. RollStore: Hybrid Onchain-Offchain Data Indexing for Blockchain Applications. *IEEE Transactions on Knowledge and Data Engineering* (2024).
 - [32] Ralph C. Merkle. 1980. Protocols for Public Key Cryptosystems. In *Proceedings of the 1980 IEEE Symposium on Security and Privacy, Oakland, California, USA, April 14–16, 1980*. IEEE Computer Society, 122–134. <https://doi.org/10.1109/SP.1980.10006>
 - [33] Dimitrios Papadopoulos, Stavros Papadopoulos, and Nikos Triandopoulos. 2014. Taking authenticated range queries to arbitrary dimensions. In *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security*. 819–830.
 - [34] Dimitrios Papadopoulos, Charalampos Papamanthou, Roberto Tamassia, and Nikos Triandopoulos. 2015. Practical authenticated pattern matching with optimal proof size. *Proceedings of the VLDB Endowment* 8, 7 (2015), 750–761.
 - [35] Mohamed Sabt, Mohammed Achemlal, and Abdelmadjid Bouabdallah. 2015. Trusted execution environment: what it is, and what it is not. In *2015 IEEE Trustcom/BigDataSE/ISPA, Vol. 1*. IEEE, 57–64.
 - [36] Srinath Setty, Justin Thaler, and Riad Wahby. 2024. Unlocking the lookup singularity with Lasso. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 180–209.
 - [37] Rohit Sinha and Mihai Christodorescu. 2018. Veritasdb: High throughput key-value store with integrity. *Cryptology ePrint Archive* (2018).
 - [38] Roberto Tamassia. 2003. Authenticated data structures. In *Algorithms-ESA 2003: 11th Annual European Symposium, Budapest, Hungary, September 16–19, 2003. Proceedings 11*. Springer, 2–5.
 - [39] Transaction Processing Council. 2023. TPC-H Benchmark. <http://www.tpc.org/tpch/>. Accessed: 2023-xx-xx.
 - [40] Tiacheng Xie, Jiaheng Zhang, Yupeng Zhang, Charalampos Papamanthou, and Dawn Song. 2019. Libra: Succinct zero-knowledge proofs with optimal prover computation. In *Advances in Cryptology—CRYPTO 2019: 39th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18–22, 2019, Proceedings, Part III* 39. Springer, 733–764.
 - [41] Yin Yang, Dimitris Papadias, Stavros Papadopoulos, and Panos Kalnis. 2009. Authenticated join processing in outsourced databases. In *Proceedings of the 2009 ACM SIGMOD International Conference on Management of data*. 5–18.
 - [42] Cong Yue, Tien Tuan Anh Dinh, Zhongle Xie, Meihui Zhang, Gang Chen, Beng Chin Ooi, and Xiaokui Xiao. 2022. GlassDB: An efficient verifiable ledger database system through transparency. *arXiv preprint arXiv:2207.00944* (2022).
 - [43] Cong Yue, Meihui Zhang, Changhao Zhu, Gang Chen, Dumitrel Loghin, and Beng Chin Ooi. 2023. VeriBench: Analyzing the Performance of Database Systems with Verifiability. *Proceedings of the VLDB Endowment* 16, 9 (2023), 2145–2157.
 - [44] Zcash. [n.d.]. halo2. <https://github.com/zcash/halo2>
 - [45] Zcash. [n.d.]. Halo2 Protocol. <https://zcash.github.io/halo2/design/protocol.html>
 - [46] Zcash. [n.d.]. PLONKish Arithmetization. <https://zcash.github.io/halo2/concepts/arithmetization.html>
 - [47] Yupeng Zhang, Daniel Genkin, Jonathan Katz, Dimitrios Papadopoulos, and Charalampos Papamanthou. 2017. vSQL: Verifying arbitrary SQL queries over dynamic outsourced databases. In *2017 IEEE Symposium on Security and Privacy (SP)*. IEEE, 863–880.
 - [48] Yupeng Zhang, Daniel Genkin, Jonathan Katz, Dimitrios Papadopoulos, and Charalampos Papamanthou. 2017. A zero-knowledge version of vSQL. *Cryptology ePrint Archive* (2017).
 - [49] Yupeng Zhang, Jonathan Katz, and Charalampos Papamanthou. 2015. IntegriDB: Verifiable SQL for outsourced databases. In *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*. 1480–1491.
 - [50] Qingji Zheng, Shouhuai Xu, and Giuseppe Ateniese. 2012. Efficient query integrity for outsourced dynamic databases. In *Proceedings of the 2012 ACM Workshop on Cloud computing security workshop*. 71–82.
 - [51] Wenting Zheng, Ankur Dave, Jethro G Beekman, Raluca Ada Popa, Joseph E Gonzalez, and Ion Stoica. 2017. Opaque: An oblivious and encrypted distributed analytics platform. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*. 283–298.

环境。2023 年 IEEE 第 43 届分布式计算系统国际会议 (ICDCS)。IEEE, 1029–1032。

[3] Arvind Arasu、Ken Eguro、Raghav Kaushik、Donald Kossmann、Pingfan Meng、Vineet Pandey 和 Ravi Ramamurthy。2017. Concerto: 高并发具有完整性的键值存储。载于 2017 年 ACM 国际数据管理会议论文集。251–266。

[4] Sumeet Bajaj 和 Radu Sion。2013. CorrectDB: 具有实用查询认证的 SQL 引擎。VLDB Endowment 会议论文集 6, 7 (2013), 529–540. [5] Eli Ben-Sasson、Iddo Bentov、Yinon Horesh 和 Michael Riabzev。2019. 可扩展零知识, 无需可信设置。发表于《密码学进展——CRYPTO 2019》: 第 39 届国际密码学年会, 美国加利福尼亚州圣巴巴拉, 2019 年 8 月 18 日至 22 日, 会议论文集, 第三部分 39。Springer, 701–732。

[6] Eli Ben-Sasson、Alessandro Chiesa、Daniel Genkin、Eran Tromer 和 Madars Virza。2013. SNARKs for C: Verifying program executions succinctly and in zero knowledge. In Annual cryptology conference. Springer, 90–108. [7] Eli Ben-Sasson、Alessandro Chiesa、Eran Tromer 和 Madars Virza。2017. Scalable zero knowledge via cycles of elliptic curves. Algorithmica 79, 4 (2017), 1102–1160。

[8] 乔纳森·布特尔特、安德里亚·塞鲁利、皮罗斯·柴多斯、詹斯·格罗斯和克里斯托夫·佩蒂特。2016. 离散对数设置下算术电路的高效零知识论证。载于《密码学进展——EUROCRYPT 2016: 第 35 届密码技术理论与应用国际年会论文集》, 奥地利维也纳, 2016 年 5 月 8-12 日, 第二部分, 第 35 卷。Springer 出版社, 第 327-357 页。

[9] Sean Bowe、Jack Grigg 和 Daira Hopwood。2019 年。无需可信设置的递归证明组合。密码学电子预印本库 (2019)。

[10] Benedikt Bünz、Jonathan Bootle、Dan Boneh、Andrew Poelstra、Pieter Wuille 和 Greg Maxwell。2018. Bulletproofs: Short proofs for confidential transactions and more. In 2018 IEEE symposium on security and privacy (SP). IEEE, 315–334。

[11] Benedikt Bünz、Alessandro Chiesa、Pratyush Mishra 和 Nicholas Spooner。2020. 从累积模式递归证明组合。在理论中密码学: 第 18 届国际会议, TCC 2020, 美国北卡罗来纳州达勒姆, 2020 年 11 月 16 日至 19 日, 会议论文集, 第二部分 18。Springer, 1-18。

[12] Ji-Won Byun 和 Ninghui Li。2008. 基于目的的关系数据库系统隐私保护访问控制. VLDB Journal 17 (2008), 603–619. [13] Alessandro Chiesa, Yuncong Hu, Mary Maller, Pratyush Mishra, Noah Vesely, 以及 Nicholas Ward。2020 年。Marlin: 使用通用和预处理 zkSNARKs 可更新的 SRS。在《密码学进展——EUROCRYPT 2020: 第 39 届密码技术理论与应用国际年会论文集》, 克罗地亚萨格勒布, 2020 年 5 月 10 日至 14 日, 会议论文集, 第一部分 39。Springer, 738–768。

[14] 塔尔·德雷、本杰明·奥伦巴赫、维克多·卡罗里诺、迦勒·格伦、迈克尔·考夫曼、Jonathan Klein、Rishad Islam Shanto 和 Henry F Korth。2023 年。《将零知识扩展到可验证数据库》。载于第一届可验证数据库系统研讨会论文集。1–9。

[15] Cynthia Dwork。2006. 差分隐私。载于《国际自动机、语言和程序设计研讨会论文集》。Springer 出版社, 第 1-12 页。[16] Amos Fiat 和 Adi Shamir。1986. 如何证明自己: 实用解决方案 身份识别和签名问题。见: 密码技术理论与应用会议论文集。Springer 出版社, 186-194 页。

[17] Uriel Fiege、Amos Fiat 和 Adi Shamir。1987 年。身份的零知识证明。在第十九届 ACM 计算理论年会论文集中。210–217。

[18] Georg Fuchsbauer、Eike Kiltz 和 Julian Loss。2018. 代数群模型及其应用。见《密码学进展——CRYPTO 2018: 第 38 届国际密码学年会, 美国加利福尼亚州圣巴巴拉, 2018 年 8 月 19 日至 23 日, 会议论文集, 第二部分 38》。Springer, 33–62。

[19] Ariel Gabizon 和 Zachary J Williamson。2020. plookup: 一种简化的查找表多项式协议。密码学电子预印本库 (2020)。[20] Shafi Goldwasser、Yael Tauman Kalai 和 Guy N Rothblum。2015. 委托计算: 面向麻瓜的交互式证明。ACM 杂志 (JACM) 62, 4 (2015), 1–64。

[21] S. Goldwasser、S. Micali 和 C. Rackoff。1985. 知识复杂性交互式证明系统。第 17 届 ACM 计算理论年会论文集, 1985 年 5 月 6 日至 8 日, 美国罗德岛州普罗维登斯。

[22] Shafi Goldwasser 和 Michael Sipser。1986. 私人硬币与公共硬币交互式证明系统。第十八届 ACM 计算理论年会论文集。59-68。

[23] Michael T Goodrich、Michael Mitzenmacher、Olga Ohrimenko 和 Roberto Tamassia。2012. 通过无状态不经意 RAM 模拟实现隐私保护的群组数据访问。载于第二十三届 ACM-SIAM 离散算法年会论文集。SIAM, 157–167。

[24] Binbin Gu 和 Faisal Nawab。2024. zk-oracle: 可信的链下计算和去中心化应用程序的存储。分布式和并行数据库 (2024), 1-24。

[25] 顾彬彬, 阿布舍克·辛格, 周一楠, 方俊成, 费萨尔·纳瓦布。2023 年。链上机器学习: 区块链上机器学习的案例和分类。2023 年 IEEE 区块链与加密货币国际会议 (ICBC)。IEEE, 1–18。

[26] Noah Johnson、Joseph P Near 和 Dawn Song。2018. 迈向实用微分 SQL 查询的隐私性。VLDB Endowment 会议论文集 11, 5 (2018), 526–539。

[27] 凯特·基希、杰森·安德森、卓震、皮埃尔·里托、保罗·鲁斯、丹·斯坦齐奥内、Mert Cevik、Jacob Colleran、Haryadi S. Gunawi、Cody Hammock、Joe Mambretti、Alexander Barnes、François Halbach、Alex Rocha 和 Joe Stubbs。2020 年。《从 Chameleon 测试平台汲取的经验教训》。载于 2020 年 USENIX 年度技术会议 (USENIX ATC '20) 会议录。USENIX 协会。

[28] Abhiram Kothapalli、Srinath Setty 和 Ioanna Tzialla。2022. Nova: 递归折叠方案中的零知识论证。载于《国际密码学年会论文集》。Springer 出版社, 第 359-388 页。

[29] Duc V Le、Lizzy Tengana Hurtado、Adil Ahmad、Mohsen Minaei、Byoungyoung Lee 和 Aniket Kate。2020. 两棵树的故事: 一棵树写作, 另一棵树阅读。隐私增强技术会议论文集 (2020)。

[30] Xiling Li、Chenkai Weng、Yongxin Xu、Xiao Wang, and Jennie Rogers。2023. ZKSQL: 基于零知识证明的可验证且高效的查询评估。VLDB 基金会论文集 16, 8 (2023), 1804–1816。

[31] 林齐, 顾彬彬, 费萨尔·纳瓦布。2024. RollStore: 链上链下混合区块链应用的数据索引。IEEE 知识与数据工程学报 (2024)。

[32] Ralph C. Merkle。1980. 公钥密码系统协议。载于 Pro-1980 年 IEEE 安全与隐私研讨会论文集, 美国加利福尼亚州奥克兰, 1980 年 4 月 14-16 日。IEEE 计算机学会, 122-134 页。https://doi.org/10.1109/SP.1980.10006

[33] 迪米特里奥斯·帕帕多普洛斯、斯塔夫罗斯·帕帕多普洛斯和尼科斯·特里安多普洛斯。2014 年。将经过认证的范围查询扩展到任意维度。在会议论文集中 2014 年 ACM SIGSAC 计算机与通信安全会议。819–830。

[34] 迪米特里奥斯·帕帕多普洛斯 (Dimitrios Papadopoulos)、查拉兰波斯·帕帕曼索 (Charalampos Papamanthou) 和伯托·塔马西亚 (Roberto Tamassia)。2015. 具有最优证明大小的实用认证模式匹配。VLDB Endowment 会议录 8, 7 (2015), 750–761。

[35] 穆罕默德·萨巴特、穆罕默德·阿赫姆拉尔和阿卜杜勒马吉德·布阿卜杜拉。2015 年。可信执行环境: 它是什么, 它不是什么。2015 年 IEEE Trustcom/BigDataSE/Ispas, 第 1 卷。IEEE, 57–64。

[36] Srinath Setty、Justin Thaler 和 Riad Wahby。2024 年。《解锁查找》Lasso 的奇点性。在密码技术理论与应用年度国际会议上。Springer, 180–209。

[37] Rohit Sinha 和 Mihai Christodorescu。2018. Veritasdb: 具有完整性的高吞吐量键值存储。密码学电子预印本库 (2018)。[38] Roberto Tamassia。2003. 认证数据结构。载于 Algorithms-ESA 2003: 第 11 届欧洲年度研讨会, 匈牙利布达佩斯, 2003 年 9 月 16-19 日。会议论文集 11。Springer, 2–5。

[39] 交易处理委员会。2023 年。TPC-H 基准测试。http://www.tpc.org/tpch/。访问日期: 2023-xx-xx。[40] 谢天成、张家恒、张宇鹏、Charalampos Papamanthou, 和 Dawn Song。2019. Libra: 具有最优证明器计算的简洁零知识证明。载于《密码学进展——CRYPTO 2019: 第 39 届国际密码学年会, 美国加利福尼亚州圣巴巴拉, 2019 年 8 月 18 日至 22 日, 会议论文集, 第三部分 39》。Springer, 733–764。

[41] Yin Yang, 迪米特里斯·帕帕迪阿斯, 斯塔夫罗斯·帕帕多普洛斯, 帕诺斯·卡尔尼斯。2009 年。外包数据库中的身份验证连接处理。载于 2009 年 ACM SIGMOD 数据管理国际会议论文集。5–18。

[42] Cong Yue、Tien Tuan Anh Dinh、Zhongle Xie、Meihui Zhang, Gang Chen, Beng Chin Ooi 和 Xiaokui Xiao。2022. GlassDB: 一种通过透明性实现高效可验证账本数据库系统。arXiv 预印本 arXiv:2207.00944 (2022)。

[43] Cong Yue、Meihui Zhang、Changhao Zhu、Gang Chen、Dumitrel Loghin, and Beng Chin Ooi。2023. VeriBench: 利用可验证性分析数据库系统的性能。VLDB Endowment 会议论文集 16, 9 (2023), 2145–2157。[44] Zcash. [无日期]. halo2. https://github.com/zcash/halo2 [45] Zcash. [无日期]. Halo2 协议. https://zcash.github.io/halo2/design/protocol.html [46] Zcash. [无日期]. PLONKish 算术. https://zcash.github.io/halo2/concepts/arithmeticization.html [47] Yupeng Zhang、Daniel Genkin、Jonathan Katz、Dimitrios Papadopoulos 和 Charalampos Papamanthou。2017. vSQL: 验证动态外包数据库上的任意 SQL 查询。载于 2017 年 IEEE 安全与隐私研讨会 (SP)。IEEE, 863–880。

[48] 张宇鹏、丹尼尔·根金、乔纳森·卡茨、迪米特里奥斯·帕帕多普洛斯和 Charalampos Papamanthou。2017. vSQL 的零知识版本。密码学电子预印本库 (2017)。

[49] Yupeng Zhang、Jonathan Katz 和 Charalampos Papamanthou。2015. IntegriDB: 外包数据库的可验证 SQL。第 22 届 ACM SIGSAC 计算机与通信安全会议论文集。1480–1491。

[50] Qingji Zheng、Shouhuai Xu, and Giuseppe Ateniese。2012. Efficient query integrity 针对外包动态数据库。载于 2012 年 ACM 云计算安全研讨会论文集, 第 71-82 页。

[51] 郑文婷、Ankur Dave、Jethro G Beekman、Raluca Ada Popa、Joseph E Gonzalez 和 Ion Stoica。2017. Opaque: 一个不感知且加密的分布式分析平台。第 14 届 USENIX 网络系统设计与实现研讨会 (NSDI 17) 论文集。283–298。

- [52] Wenchao Zhou, Yifan Cai, Yanqing Peng, Sheng Wang, Ke Ma, and Feifei Li. 2021. Veridb: An sgx-based verifiable database. In *Proceedings of the 2021 International Conference on Management of Data*. 2182–2194.
- [53] Yizheng Zhu, Yuncheng Wu, Zhaojing Luo, Beng Chin Ooi, and Xiaokui Xiao. 2023. Secure and verifiable data collaboration with low-cost zero-knowledge proofs. *arXiv preprint arXiv:2311.15310* (2023).

[52] Wenchao Zhou, Yifan Cai, Yanqing Peng, Sheng Wang, Ke Ma, and Feifei Li. 2021. Veridb: 一个基于 sgx 的可验证数据库。载于 2021 年国际数据管理会议论文集。2182–2194。

[53] Yizheng Zhu, Yuncheng Wu, Zhaojing Luo, Beng Chin Ooi, and Xiaokui Xiao. 2023. 使用低成本零知识证明实现安全可验证的数据协作。arXiv 预印本 arXiv:2311.15310 (2023)。